# Competitive Programming Template

Tired But Not Retired

July 2025

## Contents


**1 String Algorithms** **4**
- 1.1 Prefix Function (KMP) . . . 4
- 1.2 KMP . . . 4
- 1.3 Z Function . . . 4
- 1.4 Manacher . . . 4
- 1.5 Trie . . . 5
- 1.6 Hashing . . . 5
- 1.7 2D Hashing . . . 6
- 1.8 Hashing With Segment Tree . . . 7
- 1.9 Double Hashing . . . 8
- 1.10 Unordered Double Hashing . . . 9
- 1.11 Suffix Array O(N LOG2(N)) . . . 10
- 1.12 Suffix Array O(N LOG(N)) . . . 10
- 1.13 LCP Array . . . 11
- 1.14 Suffix Automaton . . . 11
  - 1.14.1 Usage example . . . 13
- 1.15 Aho-Corasick automaton . . . 13
  - 1.15.1 Usage example . . . 14

**2 Graph Algorithms** **14**
- 2.1 Dijkstra . . . 14
- 2.2 Bellman-Ford (Detect Negative Cycles) . . . 15
- 2.3 Floyd . . . 15
- 2.4 DSU . . . 15
- 2.5 DSU with Rollback . . . 16
- 2.6 LCA . . . 16
- 2.7 SCC Kosaraju . . . 17
- 2.8 SCC Tarjan . . . 18
- 2.9 Finding Bridges in Graph . . . 18
- 2.10 Finding articulation points . . . 19
- 2.11 Cycle Detection (Undirected Graph) . . . 19
- 2.12 Cycle Detection (Directed Graph) . . . 20
- 2.13 Get cycle in graph . . . 20
- 2.14 Tree Diameter . . . 21
- 2.15 Topological Sort . . . 21
- 2.16 HLD . . . 22

**3 Range queries** **23**
- 3.1 Prefix and Partial 2D . . . 23
- 3.2 Policy Data Structures . . . 23
- 3.3 BIT . . . 24
- 3.4 BIT 2D . . . 24
- 3.5 Sparse Table . . . 25
- 3.6 Segment Tree . . . 25
- 3.7 Segment Tree Lazy . . . 26
- 3.8 Merge Sort Segment Tree . . . 27
- 3.9 Segment Tree Subarray queries . . . 28

3.10 Sparse Segment Tree . . . 28
3.11 Segment Tree 2D . . . 29
3.12 Persistent Segment Tree . . . 31
3.13 Mo . . . 32
3.14 Sqrt Decomposition . . . 33

**4 Math** **33**
4.1 Operaions under mod . . . 33
4.2 is Prime . . . 34
4.3 is Prime Miller-Rabin . . . 34
4.4 Sieve . . . 35
4.5 SPF . . . 35
4.6 Prime factorizations . . . 36
4.7 Prime factorizations + SPF . . . 36
4.8 Prime factorization (Big numbers) . . . 36
4.8.1 Usage example . . . 37
4.9 Segmented Sieve . . . 37
4.10 nCr & nPr . . . 38
4.11 nCr & nPr with Precompute . . . 38
4.12 nCr with mod – Lucas Theorem . . . 39
4.12.1 Usage example . . . 40
4.13 Mobius Function (M(n)) . . . 40
4.14 Matrix Exponentiation . . . 40
4.15 Matrix Exponentiation Many Queries . . . 41
4.16 FFT . . . 42
4.17 NTT with MOD . . . 43
4.18 Check Primitive Root . . . 44
4.19 Solve equation . . . 45
4.20 Gaussian elimination . . . 45
4.20.1 Usage example . . . 46

**5 Geometry** **46**
5.1 define some struct . . . 46
5.2 relation between two lines . . . 46
5.3 distance from point to (line — segment) . . . 47
5.4 Check if point on segment . . . 47
5.5 Check if two segments intersect . . . 47
5.6 Intersection between two lines . . . 48
5.7 Angle between two lines . . . 48
5.8 Find points at dist a from p1 and dist b from p2 . . . 48
5.9 Triangle area . . . 48
5.9.1 Triangle area from points . . . 48
5.9.2 Triangle area from sides . . . 49
5.10 Classify triangle & Angle type . . . 49
5.11 Centroid of triangle . . . 49
5.12 Circle inside triangle . . . 49
5.13 Circle outside triangle . . . 49
5.14 Check if point inside triangle . . . 50
5.15 Point to circle relation . . . 50
5.16 Circle to Circle relation . . . 50
5.17 intersection between line & circle . . . 50
5.18 Tangents from point to circle . . . 51
5.19 intersection area between two circles . . . 51
5.20 Polygon area . . . 52
5.21 Polygon perimeter . . . 52
5.22 Check Polygon is convex . . . 52
5.23 Check point in polygon . . . 52
5.24 Clip convex polygon . . . 53
5.25 Convex Hull . . . 53

**6 Big Numbers** **54**
6.1 add . . . 54

6.2 subtract . . . 54
6.3 compare . . . 55
6.4 multiply with int . . . 55
6.5 divide by int . . . 55
6.6 int prime MOD . . . 56
6.7 Mod with big number . . . 56

**7 Applications** **56**
7.1 Binary Trie with XOR . . . 56
7.2 Longest increasing subsequence . . . 58
7.3 Segment Tree with Hashing . . . 58
7.4 Distinct Elements in range . . . 60
7.4.1 Usage Example . . . 61
7.5 Number of subarrays in intervals with (add & remove) interval . . . 61
7.6 Get the L, R of the longest palindrome for many queries . . . 62
7.7 Most frequent element in L, R in array . . . 65
7.8 Convert from base to another . . . 66
7.9 Halazona . . . 67
7.10 Grundy with fixed moves . . . 68

**8 Tricks** **69**
8.1 XOR numbers from 1 to R . . . 69
8.2 XOR from 1 to $R$, skipping numbers where $N \bmod 2^x \neq y$ . . . 69
8.3 XOR from 1 to $R$ with inc $X$ . . . 69
8.4 AND For range L to R . . . 69
8.5 Random number in Range . . . 70
8.6 Base-K Mask . . . 70
8.7 Generate Formula for $\sum_{i=1}^{n} i^k$ . . . 70

**9 Summation Formulas** **72**
9.1 Natural Number Sums . . . 72
9.2 Even & Odd Number sums . . . 72
9.3 Arithmetic Sequence Sum . . . 72
9.4 Geometric Sequence Sum . . . 72

# 1 String Algorithms

## 1.1 Prefix Function (KMP)

```cpp
vector<int> prefix_function(string s) {
    int n = s.size();
    vector<int> p(n);
    for (int i = 1; i < n; i++) {
        int j = p[i - 1];
        while (j && s[i] != s[j]) j = p[j - 1];
        if (s[i] == s[j]) ++j;
        p[i] = j;
    }
    return p;
}
```

## 1.2 KMP

```cpp
vector<int> kmp(const vector<int>& text, const vector<int>& pattern) {
    int n = text.size(), m = pattern.size();
    vector<int> pi = prefix_function(pattern);
    vector<int> positions;
    int j = 0;
    for (int i = 0; i < n; i++) {
        while (j > 0 && text[i] != pattern[j])
            j = pi[j - 1];
        if (text[i] == pattern[j]) j++;
        if (j == m) {
            positions.push_back(i - m + 1);
            j = pi[j - 1];
        }
    }
    return positions;
}
```

## 1.3 Z Function

```cpp
vector<int> z_function(string s) {
    int n = s.size();
    vector<int> z(n);
    z[0] = n;
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) z[i] = min(z[i - l], r - i + 1);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            l = i, r = i + z[i], z[i]++;
    }
    return z;
}
```

## 1.4 Manacher

```cpp
struct Manacher {
  vector<int> p[2];
  // p[1][i] = (max odd length palindrome centered at i) / 2 [floor division]
  // p[0][i] = same for even, it considers the right center
  // e.g. for s = "abbabba", p[1][3] = 3, p[0][2] = 2
  Manacher(string s) {
```

```
    int n = s.size();
    p[0].resize(n + 1);
    p[1].resize(n);
    for (int z = 0; z < 2; z++) {
      for (int i = 0, l = 0, r = 0; i < n; i++) {
        int t = r - i + !z;
        if (i < r) p[z][i] = min(t, p[z][l + t]);
        int L = i - p[z][i], R = i + p[z][i] - !z;
        while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
          p[z][i]++, L--, R++;
        if (R > r) l = L, r = R;
      }
    }
  }
  bool is_palindrome(int l, int r) {
    int mid = (l + r + 1) / 2, len = r - l + 1;
    return 2 * p[len % 2][mid] + len % 2 >= len;
  }
};
```

### 1.5 Trie

```
struct Node {
    Node* child[26];
    bool end;
    Node() {
        memset(child, NULL, sizeof(child));
        end = false;
    }
};

struct Trie {
    Node* root;
    Trie() {
        root = new Node();
    }
    void add(string s) {
        Node* cur = root;
        for (auto c : s) {
            if (!cur->child[c - 'a']) cur->child[c - 'a'] = new Node();
            cur = cur->child[c - 'a'];
        }
        cur->end = true;
    }
    bool find(string s) {
        Node* cur = root;
        for (auto c : s) {
            if (!cur->child[c - 'a']) return false;
            cur = cur->child[c - 'a'];
        }
        return cur->end;
    }
};
```

### 1.6 Hashing

```
struct Hash {
    int MOD = 1e9 + 9;
    int P = 31;
```

```
    vector<int> pow, hash;

    int add(int a, int b) {
        return (1ll * a + b) % MOD;
    }

    int sub(int a, int b) {
        return (1ll * a - b + MOD) % MOD;
    }

    int mul(int a, int b) {
        return (1ll * a * b) % MOD;
    }

    Hash(string& s) {
        int n = s.size();
        pow.push_back(1);
        hash.push_back(0);

        for (int i = 1; i <= n; i++) {
            pow.push_back(mul(pow.back(), P));
            hash.push_back(add(mul(hash.back(), P), s[i - 1]));
        }
    }

    int get(int l, int r) {
        return sub(hash[r + 1], mul(hash[l], pow[r - l + 1]));
    }

    int merge(int left, int right, int right_sz) {
        return add(mul(left, pow[right_sz]), right);
    }
};
```

### 1.7 2D Hashing

```
struct Hashing {
    vector<vector<int>> hs;
    vector<int> PWX, PWY;
    int n, m;
    static const int PX = 3731, PY = 2999, mod = 998244353;
    Hashing() {}
    Hashing(vector<string>& s) {
        n = (int)s.size(), m = (int)s[0].size();
        hs.assign(n + 1, vector<int>(m + 1, 0));
        PWX.assign(n + 1, 1);
        PWY.assign(m + 1, 1);
        for (int i = 0; i < n; i++) PWX[i + 1] = 1LL * PWX[i] * PX % mod;
        for (int i = 0; i < m; i++) PWY[i + 1] = 1LL * PWY[i] * PY % mod;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i + 1][j + 1] = s[i][j] - 'a' + 1;
            }
        }
        for (int i = 0; i <= n; i++) {
            for (int j = 0; j < m; j++) {
                hs[i][j + 1] = (hs[i][j + 1] + 1LL * hs[i][j] * PY % mod) %
                    mod;
            }
        }
```

```
        for (int i = 0; i < n; i++) {
            for (int j = 0; j <= m; j++) {
                hs[i + 1][j] = (hs[i + 1][j] + 1LL * hs[i][j] * PX % mod) %
                    mod;
            }
        }
    }
    int get_hash(int x1, int y1, int x2, int y2) {  // 1-indexed
        assert(1 <= x1 && x1 <= x2 && x2 <= n);
        assert(1 <= y1 && y1 <= y2 && y2 <= m);
        x1--;
        y1--;
        int dx = x2 - x1, dy = y2 - y1;
        return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1] * PWY[dy] % mod + mod)
            % mod -
                1LL * (hs[x1][y2] - 1LL * hs[x1][y1] * PWY[dy] % mod + mod)
                    % mod * PWX[dx] % mod + mod) %
                mod;
    }
    int get_hash() {
        return get_hash(1, 1, n, m);
    }
};
```

### 1.8 Hashing With Segment Tree

```
const int N = 2e5 + 1;
const int M = 1e9 + 7;
const int P = 31;
int pw[N];

int add(int a, int b) {
    return (1ll * a + b) % M;
}

int mul(int a, int b) {
    return (1ll * a * b) % M;
}

int sub(int a, int b) {
    return (1ll * a - b + M) % M;
}

int seg[N * 4], lazy[4 * N], nums[N], n, m, k;
int merge(int left, int right, int rsize) {
    return add(right, mul(left, pw[rsize]));
}

int getHash(int d, int len) {
    if (len == 0) return 0;

    int ret = getHash(d, len >> 1);
    ret = merge(ret, ret, len >> 1);
    if (len & 1) ret = merge(ret, d, 1);

    return ret;
}

void push(int i, int low, int high) {
    if (lazy[i] == -1) return;
```

```
    if (low < high) {
        lazy[2 * i] = lazy[i];
        lazy[2 * i + 1] = lazy[i];
    }
    seg[i] = getHash(lazy[i], high - low + 1);
    lazy[i] = -1;
}

void build(int i, int low, int high) {
    if (low == high) {
        seg[i] = nums[low];
        return;
    }

    int mid = low + high >> 1;
    build(2 * i, low, mid);
    build(2 * i + 1, mid + 1, high);
    seg[i] = merge(seg[2 * i], seg[2 * i + 1], high - mid);
}

pair<int, int> get(int i, int low, int high, int l, int r) {
    push(i, low, high);
    if (high < l || r < low) return {0, 0};
    if (l <= low && r >= high) return {seg[i], high - low + 1};
    int mid = low + high >> 1;

    pair<int, int> left = get(2 * i, low, mid, l, r);
    pair<int, int> right = get(2 * i + 1, mid + 1, high, l, r);

    return {merge(left.first, right.first, right.second), left.second +
        right.second};
}

int get(int l, int r) {
    return get(1, 1, n, l, r).first;
}

void update(int i, int low, int high, int l, int r, int d) {
    push(i, low, high);
    if (high < l || r < low) return;
    if (l <= low && r >= high) {
        lazy[i] = d;
        push(i, low, high);
        return;
    }

    int mid = low + high >> 1;
    update(2 * i, low, mid, l, r, d);
    update(2 * i + 1, mid + 1, high, l, r, d);
    seg[i] = merge(seg[2 * i], seg[2 * i + 1], high - mid);
}

void update(int l, int r, int d) {
    return update(1, 1, n, l, r, d);
}
```

### 1.9 Double Hashing

```
const int N = 1e5 + 5, MOD1 = 1e9 + 7, MOD2 = 1e9 + 9, LOG = 18;
int pw1[N], inv1[N], pw2[N], inv2[N], BASE = 8293;
```

```
3
4  int fix(ll x, int M) {
5      return (x % M + M) % M;
6  }
7
8  int fpow(int a, int b, int mod) {
9      if (!b) return 1;
10     int ret = fpow(a, b >> 1, mod);
11     ret = fix(1ll * ret * ret, mod);
12     if (b & 1) ret = fix(1ll * ret * a, mod);
13     return ret;
14 }
15
16 void init() {
17     static bool done = false;
18     if (done) return;
19     done = true;
20     pw1[0] = inv1[0] = pw2[0] = inv2[0] = 1;
21     int iv1 = fpow(BASE, MOD1 - 2, MOD1);
22     int iv2 = fpow(BASE, MOD2 - 2, MOD2);
23     for (int i = 1; i < N; ++i) {
24         pw1[i] = fix(1ll * pw1[i - 1] * BASE, MOD1);
25         pw2[i] = fix(1ll * pw2[i - 1] * BASE, MOD2);
26         inv1[i] = fix(1ll * inv1[i - 1] * iv1, MOD1);
27         inv2[i] = fix(1ll * inv2[i - 1] * iv2, MOD2);
28     }
29 }
30
31 struct Hash {
32     vector<pair<int, int>> pre;
33     void build(const string &s) {
34         init();
35         pre.assign(s.size() + 1, {0, 0});
36         for (int i = 0; i < s.size(); i++) {
37             pre[i + 1] = make_pair(fix(1ll * pw1[i] * s[i] + pre[i].first,
                   MOD1),
38                                    fix(1ll * pw2[i] * s[i] + pre[i].second,
                                       MOD2));
39         }
40     }
41     pair<int, int> getRange(int l, int r) const { // 0-based
42         return make_pair(fix(1ll * inv1[l] * (pre[r + 1].first -
               pre[l].first), MOD1),
43                          fix(1ll * inv2[l] * (pre[r + 1].second -
                               pre[l].second), MOD2));
44     }
45 };
```

### 1.10 Unordered Double Hashing

```
1  const int N = 1e5 + 5, MOD1 = 1e9 + 7, MOD2 = 1e9 + 9, LOG = 18;
2  int pw1[N], pw2[N], BASE = 8293;
3
4  int fix(ll x, int M) {
5      return (x % M + M) % M;
6  }
7
8  void init() {
9      static bool done = false;
10     if (done) return;
```

```
    done = true;
    pw1[0] = pw2[0] = 1;
    for (int i = 1; i < N; ++i) {
        pw1[i] = fix(1ll * pw1[i - 1] * BASE, MOD1);
        pw2[i] = fix(1ll * pw2[i - 1] * BASE, MOD2);
    }
}

struct UnOrderedHash {
    vector<pair<int, int>> pre;
    void build(const string&s) {
        init();
        pre.assign(s.size() + 1, {0, 0});
        for (int i = 0; i < s.size(); i++) {
            pre[i + 1] = make_pair(fix(1ll * pw1[s[i]] + pre[i].first, MOD1),
                                   fix(1ll * pw2[s[i]] + pre[i].second,
                                       MOD2));
        }
    }
    pair<int, int> getRange(int l, int r) const { // 0-based
        return make_pair(fix(1ll * (pre[r + 1].first - pre[l].first), MOD1),
                         fix(1ll * (pre[r + 1].second - pre[l].second),
                             MOD2));
    }
};
```

### 1.11 Suffix Array O(N LOG2(N))

```
vector<int> suffixArray(const string &s) {
    int n = s.size();
    vector<int> sa(n), rank(n), tmp(n);
    for (int i = 0; i < n; i++) {
        sa[i] = i;
        rank[i] = s[i];
    }
    for (int h = 1; h < n; h *= 2) {
        auto cmp = [&](int a, int b) {
            if (rank[a] != rank[b]) return rank[a] < rank[b];
            int ra = (a + h < n) ? rank[a + h] : -1;
            int rb = (b + h < n) ? rank[b + h] : -1;
            return ra < rb;
        };
        sort(sa.begin(), sa.end(), cmp);
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; i++) {
            tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i]) ? 1 : 0);
        }
        rank = tmp;
    }
    return sa;
}
```

### 1.12 Suffix Array O(N LOG(N))

```
vector<int> buildSuffixArray(const string &s) {
    int n = s.size();
    const int alphabet = 256;
    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++) cnt[s[i]]++;
```

```
    for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i - 1]]) classes++;
        c[p[i]] = classes - 1;
    }
    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0) pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
            pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) %
                n]};
            if (cur != prev) ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}
```

### 1.13 LCP Array

```
vector<int> buildLCP(const string &s, const vector<int> &sa) {
    int n = s.size();
    vector<int> rank(n), lcp(n);
    for (int i = 0; i < n; i++) rank[sa[i]] = i;
    int h = 0;
    for (int i = 0; i < n; i++) {
        if (rank[i] > 0) {
            int j = sa[rank[i] - 1];
            while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
            lcp[rank[i]] = h;
            if (h > 0) h--;
        }
    }
    return lcp;
}
```

### 1.14 Suffix Automaton

```
struct State {
    int len, link;
    map<char, int> next;
};

const int MAXN = 2e5 + 5;
State st[2 * MAXN];
```

```
int sz, last;

void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz = 1;
    last = 0;
}

void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;

    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }

    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;

            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }

            st[q].link = st[cur].link = clone;
        }
    }

    last = cur;
}

bool is_substring(const string& s) {
    int v = 0;
    for (char c : s) {
        if (!st[v].next.count(c))
            return false;
        v = st[v].next[c];
    }
    return true;
}
long long count_different_substrings() {
    long long res = 0;
    for (int i = 1; i < sz; i++) {
        res += st[i].len - st[st[i].link].len;
    }
    return res;
}
```

### 1.14.1 Usage example

```cpp
int main() {
    string s;
    cin >> s;
    sa_init();
    for (char c : s)
        sa_extend(c);

    cout << "Total distinct substrings: " << count_different_substrings() << '\n';

    string query;
    while (cin >> query) {
         cout << (is_substring(query) ? "Yes" : "No") << '\n';
     }
    return 0;
}
```

## 1.15 Aho-Corasick automaton

```cpp
const int A = 26;         // alphabet size
const int N = 1e5 + 5;    // max total length of all patterns
int trie[N][A], link[N], out[N], term[N];
int node_cnt = 1;
void add(const string &s, int id) {
    int u = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (!trie[u][c]) trie[u][c] = node_cnt++;
        u = trie[u][c];
    }
    term[u] = id; // mark end with pattern ID (1-based)
}
void build() {
    queue<int> q;
    for (int c = 0; c < A; ++c) {
        if (trie[0][c]) q.push(trie[0][c]);
    }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int c = 0; c < A; ++c) {
            int v = trie[u][c];
            if (!v) continue;
            int f = link[u];
            while (f && !trie[f][c]) f = link[f];
            link[v] = trie[f][c];
            if (term[link[v]]) out[v] = link[v];
            else out[v] = out[link[v]];
            q.push(v);
        }
    }
}
vector<vector<int>> find_matches (const string &text, const vector<int> &pattern_lengths) {
    vector<vector<int>> res(pattern_lengths.size(), vector<int>());
    int u = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        int c = text[i] - 'a';
        while (u && !trie[u][c]) u = link[u];
```

```
        u = trie[u][c];
        int v = u;
        while (v) {
            if (term[v]) {
                int pattern_id = term[v]; // pattern index
                int len = pattern_lengths[pattern_id];
                res[pattern_id].emplace_back(i - len + 1); // starting
                    position of match
            }
            v = out[v];
        }
    }
    return res;
}
```

### 1.15.1 Usage example

```
int main() {
    string s;
    cin >> s;
    int n;
    cin >> n;
    vector<string>pat(n+1);
    vector<int>k(n+1), len(n+1);
    for (int i=1; i<=n; i++) {
        cin >> k[i] >> pat[i];
        len[i] = pat[i].size();
        add(pat[i], i);
    }
    build();
    vector<vector<int>>res = find_matches(s, len);
}
```

# 2 Graph Algorithms

## 2.1 Dijkstra

```
vector<ll> dijkstra(int start) {
    vector<ll> dist(n + 1, 1e18);
    priority_queue<pair<ll, int>, deque<pair<ll, int>>, greater<pair<ll,
        int>>> Q;
    Q.push({0, start});
    dist[start] = 0;

    while (!Q.empty()) {
        pair<ll, int> cur = Q.top();
        Q.pop();

        ll cost = cur.first;
        int node = cur.second;

        for (auto edge : adj[node]) {
            int nextNode = edge.first;
            ll nextCost = cost + edge.second;

            if (nextCost < dist[nextNode]) {
                dist[nextNode] = nextCost;
                Q.push({nextCost, nextNode});
            }
```

```
        }
    }
}
```

### 2.2 Bellman-Ford (Detect Negative Cycles)

```
// O(V.E)
vector<ll> bellman(int n, int src, vector<tuple<int,int,int>>& edges) {
    vector<ll> dist(n, 1e18);
    dist[src] = 0;
    for (int i = 0; i < n - 1; ++i)
        for (auto [u, v, w] : edges)
            if (dist[u] + w < dist[v])
                dist[v] = dist[u] + w;

    // Negative cycle check
    for (auto [u, v, w] : edges)
        if (dist[u] + w < dist[v])
            throw runtime_error("Negative Cycle");

    return dist;
}
```

### 2.3 Floyd

```
const int INF = 1e9;
int dist[N][N];
void floyd(int n) {
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
}
```

### 2.4 DSU

```
const int N = 5e5 + 1;
int parent[N], sz[N], n;

int find(int a) {
    if (parent[a] == a) return a;
    return parent[a] = find(parent[a]);
}

void join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b);
    sz[a] += sz[b];
    parent[b] = a;
}

void init() {
    for (int i = 0; i < n; i++)
        sz[i] = 1, parent[i] = i;
}
```

### 2.5 DSU with Rollback

```
const int N = 5e5 + 1;
stack<int> stck;
int sz[N], parent[N], n, m;

int find(int a) {
    if (parent[a] == a) return a;
    return parent[a] = find(parent[a]);
}

void join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return;
    sz[a] += sz[b];
    parent[b] = a;
    stck.push(b);
}

void rollback(int x) {
    while (stck.size() > x) {
        int k = stck.top();
        stck.pop();
        sz[parent[k]] -= sz[k];
        parent[k] = k;
    }
}

void init() {
    for (int i = 0; i < n; i++)
        sz[i] = 1, parent[i] = i;
    stck = stack<int>();
}
```

### 2.6 LCA

```
const int N = 2e5 + 1;
const int LOG = 30;
vector<int> tree[N];
int in[N], out[N], timer = 0;
int up[N][LOG], depth[N];

//Initialize by calling dfs(root, root) where root's parent is itself
void dfs(int node, int parent) {
    depth[node] = 1 + depth[parent];
    up[node][0] = parent;
    in[node] = ++timer;

    for (int log = 1; log < LOG; log++)
        up[node][log] = up[up[node][log - 1]][log - 1];
    for (auto& child : tree[node])
        if (child != parent) dfs(child, node);
    out[node] = ++timer;
}

bool ancestor(int u, int v) {
    return in[u] <= in[v] && out[u] >= out[v];
}

int lca(int u, int v) {
```

```
    if (ancestor(u, v)) return u;
    if (ancestor(v, u)) return v;
    for (int log = LOG - 1; log >= 0; log--)
        if (!ancestor(up[u][log], v)) u = up[u][log];
    return up[u][0];
}

int dist(int u, int v) {
    return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}

int kth(int node, int k) {
    for (int bit = LOG - 1; bit >= 0; bit--)
        if (k & (1 << bit))
            node = up[node][bit];
    return node;
}
```

## 2.7 SCC Kosaraju

```
const int N = 2e5 + 1;
vector<int> adj[N], radj[N], adj2[N], order, vis;
int id[N], sum[N], n, m, comp = 0;

void dfs1(int node) {
    vis[node] = 1;
    for (auto& next : adj[node])
        if (!vis[next]) dfs1(next);
    order.push_back(node);
}

void dfs2(int node) {
    vis[node] = 1;
    id[node] = comp;
    for (auto& next : radj[node])
        if (!vis[next]) dfs2(next);
}

void kosaraju() {
    vis.assign(n + 1, 0);
    order.clear();

    for (int node = 1; node <= n; node++)
        if (!vis[node]) dfs1(node);

    vis.assign(n + 1, 0);
    reverse(all(order));
    for (auto& node : order) {
        if (vis[node]) continue;
        ++comp;
        dfs2(node);
    }

    for (int node = 1; node <= n; node++) {
        sum[id[node]]++;
        for (auto& next : adj[node])
            if (id[node] != id[next])
                adj2[id[node]].push_back(id[next]);
    }

```

```
    for (int i = 1; i <= comp; i++) {
        sort(all(adj2[i]));
        adj2[i].erase(unique(all(adj2[i])), adj2[i].end());
    }
}
```

### 2.8 SCC Tarjan

```
const int N = 5e5 + 1;
vector<int> adj[N];
int in[N], low[N], n, timer = 0, c = 0;
bool inStack[N];
stack<int> st;
map<int, int> comp;

void dfs(int node) {
    low[node] = in[node] = ++timer;
    inStack[node] = true;
    st.push(node);

    for (auto next : adj[node]) {
        if (in[next] == 0) {
            dfs(next);
            low[node] = min(low[node], low[next]);
        } else if (inStack[next])
            low[node] = min(low[node], in[next]);
    }

    if (low[node] == in[node]) {
        ++c;
        while (true) {
            int cur = st.top();
            st.pop();
            inStack[cur] = false;
            comp[cur] = c;
            if (cur == node) break;
        }
    }
}

void tarjan() {
    for (int node = 1; node <= n; node++)
        if (in[node] == 0) dfs(node);
}
```

### 2.9 Finding Bridges in Graph

```
const int N = 2e5 + 1;
vector<int> adj[N];
set<pair<int, int>> bridges;
vector<pair<int, int>> edges;
int in[N], low[N], timer = 0, n, m;

void dfs(int node, int parent) {
    in[node] = low[node] = ++timer;

    for (auto next : adj[node]) {
        if (next == parent) continue;
        if (in[next] == 0) {
```

```
            dfs(next, node);
            low[node] = min(low[node], low[next]);
        } else
            low[node] = min(low[node], in[next]);
    }

    if (in[node] == low[node] && parent)
        bridges.insert({parent, node});
}

void getBridges() {
    for (int i = 1; i <= n; i++)
        if (in[i] == 0)
            dfs(i, 0);
}
```

### 2.10 Finding articulation points

```
const int N = 1e5 + 5;
vector<int> g[N];
bool vis[N];
int tin[N], low[N], timer;
set<int> articulation_points;

void dfs(int u, int parent = -1) {
    vis[u] = true;
    tin[u] = low[u] = timer++;
    int children = 0;

    for (int v : g[u]) {
        if (v == parent) continue;
        if (vis[v]) {
            low[u] = min(low[u], tin[v]);  // back edge
        } else {
            dfs(v, u);
            low[u] = min(low[u], low[v]);
            if (low[v] >= tin[u] && parent != -1)
                articulation_points.insert(u);
            children++;
        }
    }

    if (parent == -1 && children > 1)
        articulation_points.insert(u);
}

void find_articulation_points(int n) {
    timer = 0;
    articulation_points.clear();
    fill(vis, vis + n + 1, false);
    for (int i = 1; i <= n; i++) {
        if (!vis[i])
            dfs(i);
    }
}
```

### 2.11 Cycle Detection (Undirected Graph)

```
// O(N+M)
```

```cpp
bool dfs_cycle(int u, int parent, vector<vector<int>>& adj, vector<bool>&
    visited) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            if (dfs_cycle(v, u, adj, visited))
                return true;
        } else if (v != parent) {
            return true; // Found a back edge (cycle)
        }
    }
    return false;
}
bool has_cycle_undirected(int n, vector<vector<int>>& adj) {
    vector<bool> visited(n, false);
    for (int i = 0; i < n; ++i) {
        if (!visited[i] && dfs_cycle(i, -1, adj, visited))
            return true;
    }
    return false;
}
```

### 2.12 Cycle Detection (Directed Graph)

```cpp
bool dfs_cycle(int u, vector<vector<int>>& adj, vector<bool>& visited,
    vector<bool>& recStack) {
    visited[u] = true;
    recStack[u] = true;

    for (int v : adj[u]) {
        if (!visited[v]) {
            if (dfs_cycle(v, adj, visited, recStack))
                return true;
        } else if (recStack[v]) {
            return true; // Back edge -> cycle
        }
    }

    recStack[u] = false;
    return false;
}
bool has_cycle_directed(int n, vector<vector<int>>& adj) {
    vector<bool> visited(n, false), recStack(n, false);
    for (int i = 0; i < n; ++i) {
        if (!visited[i] && dfs_cycle(i, adj, visited, recStack))
            return true;
    }
    return false;
}
```

### 2.13 Get cycle in graph

```cpp
const int N = 2e5 + 1;
vector<int> adj[N], cycle;
int vis[N], parent[N];

void dfs_cycle(int node, int par) {
    if (vis[node] == 2) return;

```

```
    if (vis[node] == 1) {
        cycle.push_back(node);
        int cur = par;
        while (cur != node) {
            cycle.push_back(cur);
            cur = parent[cur];
        }
        return;
    }

    vis[node] = 1;
    parent[node] = par;
    for (auto next : adj[node]) {
        if (par != next)
            dfs_cycle(next, node);
    }

    vis[node] = 2;
    return;
}
```

### 2.14 Tree Diameter

```
const int N = 2e5 + 1;
vector<int> tree[N];
int down[N][2], diameter;

void dfs(int node, int parent) {
    for (auto next : tree[node]) {
        if (next == parent) continue;

        dfs(next, node);
        int h = 1 + down[next][0];

        if (h >= down[node][0]) {
            swap(down[node][0], down[node][1]);
            down[node][0] = h;
        } else
            down[node][1] = max(down[node][1], h);
    }
    diameter = max(diameter, down[node][0] + down[node][1]);
}
```

### 2.15 Topological Sort

```
const int N = 2e5 + 1;
vector<int> adj[N];
int degree[N], n;

bool topSort() {
    queue<int> Q;
    for (int node = 1; node <= n; node++)
        if (degree[node] == 0)
            Q.push(node);

    int vis = 0;
    while (!Q.empty()) {
        int node = Q.front();
        Q.pop();
```

```
        ++vis;

        for (auto next : adj[node])
            if (--degree[next] == 0)
                Q.push(next);
    }

    bool cycle = (vis < n);
    return cycle;
}
```

### 2.16 HLD

```
const int N = 1e5 + 1;
vector<int> adj[N], parent, depth, heavy, head, pos;
int timer;

int dfs(int v) {
    int size = 1;
    int max_c_size = 0;
    for (int c : adj[v]) {
        if (c != parent[v]) {
            parent[c] = v, depth[c] = depth[v] + 1;
            int c_size = dfs(c);
            size += c_size;
            if (c_size > max_c_size)
                max_c_size = c_size, heavy[v] = c;
        }
    }
    return size;
}

void decompose(int v, int h) {
    head[v] = h, pos[v] = timer++;
    if (heavy[v] != -1)
        decompose(heavy[v], h);
    for (int c : adj[v]) {
        if (c != parent[v] && c != heavy[v])
            decompose(c, c);
    }
}

vector<pair<int, int>> query_path(int a, int b) {
    vector<pair<int, int>> up, down;
    while (head[a] != head[b]) {
        if (depth[head[a]] > depth[head[b]]) {
            up.emplace_back(pos[a], pos[head[a]]);
            a = parent[head[a]];
        } else {
            down.emplace_back(pos[head[b]], pos[b]);
            b = parent[head[b]];
        }
    }
    if (depth[a] > depth[b])
        up.emplace_back(pos[a], pos[b]);
    else
        down.emplace_back(pos[a], pos[b]);
    reverse(down.begin(), down.end());
    up.insert(up.end(), down.begin(), down.end());
    return up;
```

```
}

void initHLD(int n) {
    n++;
    parent = vector<int>(n);
    depth = vector<int>(n);
    heavy = vector<int>(n, -1);
    head = vector<int>(n);
    pos = vector<int>(n);
    timer = 1;
    dfs(1);
    decompose(1, 0);
}
```

# 3 Range queries

## 3.1 Prefix and Partial 2D

```
int n, m;
cin >> n >> m;
vector<vector<ll>> grid(n + 5, vector<ll>(m + 5));
for (int r = 1; r <= n; r++)
    for (int c = 1; c <= m; c++)
        cin >> grid[r][c];

auto pre = grid;
for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= m; c++)
        pre[r][c] += pre[r][c - 1];
    for (int c = 1; c <= m; c++)
        pre[r][c] += pre[r - 1][c];
}

auto get = [&](int r1, int c1, int r2, int c2) {
    ll ans = pre[r2][c2];
    ans -= pre[r1 - 1][c2];
    ans -= pre[r2][c1 - 1];
    ans += pre[r1 - 1][c1 - 1];
    return ans;
};

vector<vector<ll>> partial(n + 5, vector<ll>(m + 5, 0));
auto apply = [&](int r1, int c1, int r2, int c2) {
    partial[r1][c1]++;
    partial[r1][c2 + 1]--;
    partial[r2 + 1][c1]--;
    partial[r2 + 1][c2 + 1]++;
};

for (int r = 1; r <= n; r++) {
    for (int c = 1; c <= m; c++)
        partial[r][c] += partial[r][c - 1];
    for (int c = 1; c <= m; c++)
        partial[r][c] += partial[r - 1][c];
}
```

## 3.2 Policy Data Structures

```cpp
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

template<typename K, typename V, typename Comp = std::less<>>
using ordered_map = tree<K, V, Comp, rb_tree_tag,
    tree_order_statistics_node_update>;

template<typename K, typename Comp = std::less<>>
using ordered_set = ordered_map<K, null_type, Comp>;

template<typename K, typename V, typename Comp = std::less_equal<>>
using ordered_multimap = tree<K, V, Comp, rb_tree_tag,
    tree_order_statistics_node_update>;

template<typename K, typename Comp = std::less_equal<>>
using ordered_multiset = ordered_multimap<K, null_type, Comp>;
```

### 3.3 BIT

```cpp
const int N = 1e5 + 1;
ll bit[N], n;

ll get(int i) {
    ll ans = 0;
    for (; i > 0; i -= i & -i)
        ans += bit[i];
    return ans;
}

ll get(int l, int r) {
    return get(r) - get(l - 1);
}

void add(int i, ll k = 1) {
    for (; i <= n; i += i & -i)
        bit[i] += k;
}
```

### 3.4 BIT 2D

```cpp
const int N = 1e3 + 1, M = 1e3 + 1;
ll tree[N][M], grid[N][M], n, m, q;

void add(int r, int c, ll v) {
    for (int i = r; i <= n; i += (i & -i))
        for (int j = c; j <= m; j += (j & -j))
            tree[i][j] += v;
}

ll get(int r, int c) {
    ll sum = 0;
    for (int i = r; i; i -= (i & -i))
        for (int j = c; j; j -= (j & -j))
            sum += tree[i][j];
    return sum;
}
```

```
17
18 ll get(int r1, int c1, int r2, int c2) {
19     return get(r2, c2) - get(r2, c1 - 1) - get(r1 - 1, c2) + get(r1 - 1, c1
       - 1);
20 }
```

### 3.5 Sparse Table

```
1  const int N = 1e5 + 1, LOG = 23;
2  int nums[N], table[N][LOG], lg[N], n;
3
4  void init() {
5      for (int i = 2; i <= n; i++)
6          lg[i] = 1 + lg[i / 2];
7  }
8
9  void build() {
10     for (int i = 0; i < n; ++i) table[i][0] = nums[i];
11     for (int log = 1; log < LOG; log++)
12         for (int i = 0; i + (1 << log) - 1 < n; i++)
13             table[i][log] = merge(table[i][log - 1], table[i + (1 << (log -
                   1))][log - 1]);
14 }
15
16 int query(int l, int r) {
17     int log = lg[r - l + 1];
18     return merge(table[l][log], table[r - (1 << log) + 1][log]);
19 }
20
21 int merge(int val1, int val2) {
22     return max(val1, val2);
23 }
```

### 3.6 Segment Tree

```
1  const int N = 2e5 + 1;
2  int nums[N], seg[N * 4], n;
3
4  void build(int i, int low, int high) {
5      if (low == high) {
6          seg[i] = nums[low];
7          return;
8      }
9      int mid = (low + high) >> 1;
10     build(2 * i, low, mid);
11     build(2 * i + 1, mid + 1, high);
12     seg[i] = seg[2 * i] + seg[2 * i + 1];
13 }
14
15 int get(int i, int low, int high, int l, int r) {
16     if (r < low || high < l) return 0;
17     if (l <= low && r >= high) return seg[i];
18     int mid = (low + high) >> 1;
19     return get(2 * i, low, mid, l, r) +
20            get(2 * i + 1, mid + 1, high, l, r);
21 }
22
23 void update(int i, int low, int high, int idx, int val) {
24     if (idx < low || idx > high) return;
```

```
    if (low == high) {
        seg[i] = val;
        return;
    }
    int mid = (low + high) >> 1;
    update(2 * i, low, mid, idx, val);
    update(2 * i + 1, mid + 1, high, idx, val);
    seg[i] = seg[2 * i] + seg[2 * i + 1];
}
```

### 3.7 Segment Tree Lazy

```
const int N = 2e5 + 1;
ll nums[N];
Node seg[N * 4];

struct Node {
    ll val, lazy;

    Node(ll _val = 0, ll _lazy = 0) {
        val = _val;
        lazy = _lazy;
    }
};

Node merge(Node a, Node b) {
    return Node(a.val + b.val);
}

void build(int i, int low, int high) {
    if (low == high) {
        seg[i] = Node(nums[low]);
        return;
    }
    int mid = (low + high) >> 1;
    build(2 * i, low, mid);
    build(2 * i + 1, mid + 1, high);
    seg[i] = merge(seg[2 * i], seg[2 * i + 1]);
}

void push(int i, int low, int high) {
    ll lazy = seg[i].lazy;
    if (lazy == 0) return;

    seg[i].val += lazy;
    if (low < high) {
        seg[2 * i].lazy += lazy;
        seg[2 * i + 1].lazy += lazy;
    }
    seg[i].lazy = 0;
}

ll get(int i, int low, int high, int l, int r) {
    push(i, low, high);
    if (high < l || r < low) return 0;
    if (l <= low && r >= high)
        return seg[i].val;

    int mid = (low + high) >> 1;
    return get(2 * i, low, mid, l, r) +
```

```
            get(2 * i + 1, mid + 1, high, l, r);
}

void update(int i, int low, int high, int l, int r, ll val) {
    push(i, low, high);
    if (high < l || r < low) return;
    if (l <= low && r >= high) {
        seg[i].lazy = val;
        push(i, low, high);
        return;
    }
    int mid = (low + high) >> 1;
    update(2 * i, low, mid, l, r, val);
    update(2 * i + 1, mid + 1, high, l, r, val);
    seg[i] = merge(seg[2 * i], seg[2 * i + 1]);
}
```

### 3.8 Merge Sort Segment Tree

```
const int N = 1e5 + 2;
vector<int> seg[4 * N];
int nums[N];

void build(int i, int low, int high) {
    if (low == high) {
        seg[i] = {nums[low]};
        return;
    }
    int mid = (low + high) / 2;
    build(2 * i, low, mid);
    build(2 * i + 1, mid + 1, high);
    merge(seg[2 * i].begin(), seg[2 * i].end(),
          seg[2 * i + 1].begin(), seg[2 * i + 1].end(),
          back_inserter(seg[i]));
}

int query_eq(int i, int low, int high, int l, int r, int val) {
    if (high < l || low > r) return 0;
    if (l <= low && high <= r) {
        return upper_bound(seg[i].begin(), seg[i].end(), val) -
               lower_bound(seg[i].begin(), seg[i].end(), val);
    }
    int mid = (low + high) / 2;
    return query_eq(2 * i, low, mid, l, r, val) +
           query_eq(2 * i + 1, mid + 1, high, l, r, val);
}

int query_lt(int i, int low, int high, int l, int r, int val) {
    if (high < l || low > r) return 0;
    if (l <= low && high <= r) {
        return lower_bound(seg[i].begin(), seg[i].end(), val) -
           seg[i].begin();
    }
    int mid = (low + high) / 2;
    return query_lt(2 * i, low, mid, l, r, val) +
           query_lt(2 * i + 1, mid + 1, high, l, r, val);
}

int query_gt(int i, int low, int high, int l, int r, int val) {
    if (high < l || low > r) return 0;
```

```
    if (l <= low && high <= r) {
        return seg[i].end() - upper_bound(seg[i].begin(), seg[i].end(), val);
    }
    int mid = (low + high) / 2;
    return query_gt(2 * i, low, mid, l, r, val) +
           query_gt(2 * i + 1, mid + 1, high, l, r, val);
}

int query_kth(int i, int low, int high, int l, int r, int k) {
    int lo = 0;
    int hi = 1e9;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        int cnt = (r - l + 1) - query_gt(i, low, high, l, r, mid);
        if (cnt < k)
            lo = mid + 1;
        else
            hi = mid;
    }
    return lo;
}
```

### 3.9 Segment Tree Subarray queries

```
struct Node {
    ll pref, suff, mx, sum;
    Node() {
        pref = suff = mx = sum = 0;
    }
};

Node merge(Node &a, Node &b) {
    Node c;
    c.sum = a.sum + b.sum;
    c.pref = max(a.pref, a.sum + b.pref);
    c.suff = max(b.suff, b.sum + a.suff);
    c.mx = max(a.mx, max(b.mx, a.suff + b.pref));
    return c;
}
```

### 3.10 Sparse Segment Tree

```
struct Node {
    ll val = -1e16;
} neutral;

struct segTree {
    segTree *left = nullptr, *right = nullptr;
    Node node = {};
    int start, end;

    segTree(int l = 0, int r = 0) : start(l), end(r) {}

    void extend() {
        if (left == nullptr) {
            int mid = start + end >> 1;
            left = new segTree(start, mid);
            right = new segTree(mid + 1, end);
        }
```

```
    }

    Node pushup(Node a, Node b) {
        Node ret;
        ret.val = max(a.val, b.val);
        return ret;
    }

    void build() {
        if (start == end) {
            node.val = -1e16;
            return;
        }
        extend();
        left->build();
        right->build();
        node = pushup(left->node, right->node);
    }

    void update(int idx, ll val) {
        if (start > idx || end < idx)
            return;
        if (start == end) {
            node.val = val;
            return;
        }
        extend();
        left->update(idx, val);
        right->update(idx, val);
        node = pushup(left->node, right->node);
    }

    Node query(int l, int r) {
        if (r < start || end < l)
            return neutral;
        extend();
        if (l <= start && end <= r)
            return node;
        Node ret = pushup(left->query(l, r), right->query(l, r));
        return ret;
    }

    ~segTree() {
        if (left == nullptr) return;
        delete left;
        delete right;
    }
};
```

### 3.11 Segment Tree 2D

```
ll SegOp(ll a, ll b) {
    return a + b;
}
ll SegDefault = 0;

struct SegmentTree2D {
    int n, m;
    vector<vector<int>> tree;

```

```
    SegmentTree2D(int _n, int _m) {
        n = _n;
        m = _m;
        tree.assign(4 * n, vector<int>(4 * m, SegDefault));
    }

    int get_y(int vx, int vy, int tly, int try_, int ly, int ry) {
        if (ly > ry)
            return SegDefault;
        if (ly == tly && try_ == ry)
            return tree[vx][vy];
        int tmy = (tly + try_) / 2;
        return SegOp(
            get_y(vx, vy * 2, tly, tmy, ly, min(ry, tmy)),
            get_y(vx, vy * 2 + 1, tmy + 1, try_, max(ly, tmy + 1), ry));
    }

    int get_x(int vx, int tlx, int trx, int lx, int rx, int ly, int ry) {
        if (lx > rx)
            return SegDefault;
        if (lx == tlx && trx == rx)
            return get_y(vx, 1, 0, m - 1, ly, ry);
        int tmx = (tlx + trx) / 2;
        return SegOp(
            get_x(vx * 2, tlx, tmx, lx, min(rx, tmx), ly, ry),
            get_x(vx * 2 + 1, tmx + 1, trx, max(lx, tmx + 1), rx, ly, ry));
    }

    int get(int lx, int ly, int rx, int ry) {
        return get_x(1, 0, n - 1, lx, rx, ly, ry);
    }

    void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int
       y, int new_val) {
        if (ly == ry) {
            if (lx == rx) {
                tree[vx][vy] = new_val;
            } else {
                tree[vx][vy] = SegOp(tree[vx * 2][vy], tree[vx * 2 + 1][vy]);
            }
        } else {
            int my = (ly + ry) / 2;
            if (y <= my)
                update_y(vx, lx, rx, vy * 2, ly, my, x, y, new_val);
            else
                update_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, x, y, new_val);
            tree[vx][vy] = SegOp(tree[vx][vy * 2], tree[vx][vy * 2 + 1]);
        }
    }

    void update_x(int vx, int lx, int rx, int x, int y, int new_val) {
        if (lx != rx) {
            int mx = (lx + rx) / 2;
            if (x <= mx)
                update_x(vx * 2, lx, mx, x, y, new_val);
            else
                update_x(vx * 2 + 1, mx + 1, rx, x, y, new_val);
        }
        update_y(vx, lx, rx, 1, 0, m - 1, x, y, new_val);
    }

```

```
    void update(int x, int y, int new_val) {
        update_x(1, 0, n - 1, x, y, new_val);
    }
};
```

### 3.12 Persistent Segment Tree

```
const int N = 1e5 + 5;

struct Node {
    int val;
    Node *left, *right;

    Node(int v = 0) : val(v), left(nullptr), right(nullptr) {}
};

class PersistentSegmentTree {
   private:
    int n;
    vector<Node*> versions;

    Node* build(int l, int r, const vector<int>& arr) {
        Node* node = new Node();
        if (l == r) {
            node->val = arr[l];
            return node;
        }
        int m = (l + r) / 2;
        node->left = build(l, m, arr);
        node->right = build(m + 1, r, arr);
        node->val = node->left->val + node->right->val;
        return node;
    }

    Node* update(Node* prev, int l, int r, int idx, int val) {
        Node* node = new Node();
        if (l == r) {
            node->val = val;
            return node;
        }
        int m = (l + r) / 2;
        if (idx <= m) {
            node->left = update(prev->left, l, m, idx, val);
            node->right = prev->right;
        } else {
            node->left = prev->left;
            node->right = update(prev->right, m + 1, r, idx, val);
        }
        node->val = node->left->val + node->right->val;
        return node;
    }

    int query(Node* node, int l, int r, int ql, int qr) {
        if (qr < l || ql > r) return 0;
        if (ql <= l && r <= qr) return node->val;
        int m = (l + r) / 2;
        return query(node->left, l, m, ql, qr) +
               query(node->right, m + 1, r, ql, qr);
    }

```

```
    public:
    PersistentSegmentTree(const vector<int>& arr) {
        n = arr.size() - 1;  // assuming 1-based indexing
        versions.push_back(build(1, n, arr));
    }

    void updateVersion(int versionIndex, int pos, int newVal) {
        Node* newRoot = update(versions[versionIndex], 1, n, pos, newVal);
        versions.push_back(newRoot);
    }

    int queryVersion(int versionIndex, int l, int r) {
        return query(versions[versionIndex], 1, n, l, r);
    }

    int versionCount() const {
        return versions.size();
    }
};

int main() {
    vector<int> arr = {0, 1, 2, 3, 4, 5};  // 1-based indexing

    PersistentSegmentTree pst(arr);

    // Version 0: original
    cout << "v0 sum[2,4] = " << pst.queryVersion(0, 2, 4) << "\n";  // 2+3+4
       = 9

    // Create version 1 with arr[3] = 10
    pst.updateVersion(0, 3, 10);
    cout << "v1 sum[2,4] = " << pst.queryVersion(1, 2, 4) << "\n";  //
       2+10+4 = 16

    // Version 0 remains unchanged
    cout << "v0 sum[2,4] = " << pst.queryVersion(0, 2, 4) << "\n";  // still
       9
}
```

### 3.13 Mo

```
const int block = 417;
struct Query {
    int l, r, id;
};

bool compare(Query& a, Query& b) {
    int ba = a.l / block, bb = b.l / block;
    if (ba != bb) return ba < bb;
    return a.r > b.r;
}

const int N = 5e6 + 1;
ll nums[N], n, q;

ll ans[N], res = 0;
void add(int i) {
    //
}

```

```
void remove(int i) {
    //
}

vector<Query> queries;
void Mo() {
    sort(all(queries), compare);

    int mo_left = 0, mo_right = -1;
    for (auto& cur : queries) {
        int l = cur.l, r = cur.r, id = cur.id;
        while (mo_left < l) remove(mo_left++);
        while (mo_left > l) add(--mo_left);
        while (mo_right < r) add(++mo_right);
        while (mo_right > r) remove(mo_right--);
        ans[id] = res;
    }
}
```

### 3.14 Sqrt Decomposition

```
struct Sqrt {
    int block_size;
    vector<int> nums;
    vector<ll> blocks;
    Sqrt(int sqrtn, vector<int> &arr) : block_size(sqrtn), blocks(sqrtn, 0) {
        nums = arr;
        for (int i = 0; i < nums.size(); i++)
            blocks[i / block_size] += nums[i];
    }

    /** O(1) update to set nums[x] to v */
    void update(int x, int v) {
        blocks[x / block_size] -= nums[x];
        nums[x] = v;
        blocks[x / block_size] += nums[x];
    }

    /** O(sqrt(n)) query for sum of [0, r) */
    ll query(int r) {
        ll res = 0;
        for (int i = 0; i < r / block_size; i++)
            res += blocks[i];
        for (int i = (r / block_size) * block_size; i < r; i++)
            res += nums[i];
        return res;
    }

    /** O(sqrt(n)) query for sum of [l, r) */
    ll query(int l, int r) { return query(r) - query(l - 1); }
};
```

# 4 Math

## 4.1 Operaions under mod

```
const int mod = 1e9 + 7;

```

```
ll add(ll a, ll b) {
    return (a + b) % mod;
}

ll sub(ll a, ll b) {
    return (a - b + mod) % mod;
}

ll mul(ll a, ll b) {
    return a * b % mod;
}

ll fpow(ll a, ll b) {
    if (!b) return 1;
    ll x = fpow(a, b >> 1);
    x = mul(x, x);
    if (b & 1) x = mul(x, a);
    return x;
}

ll divide(ll a, ll b) {
    return mul(a, fpow(b, mod - 2));
}
```

### 4.2 is Prime

```
bool isPrime(int n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    return true;
}
```

### 4.3 is Prime Miller-Rabin

```
typedef unsigned long long ull;
ull mulmod(ull a, ull b, ull mod) {
    ull res = 0;
    a %= mod;
    while (b) {
        if (b & 1) res = (res + a) % mod;
        a = (2 * a) % mod;
        b >>= 1;
    }
    return res;
}
ull powmod(ull a, ull b, ull mod) {
    ull res = 1;
    a %= mod;
    while (b) {
        if (b & 1) res = mulmod(res, a, mod);
        a = mulmod(a, a, mod);
        b >>= 1;
    }
    return res;
}
```

```
22
23 bool millerRabin(ull n, ull a) {
24     if (n % a == 0) return false;
25     ull d = n - 1;
26     while (d % 2 == 0) d /= 2;
27     ull x = powmod(a, d, n);
28     if (x == 1 || x == n - 1) return true;
29     while (d != n - 1) {
30         x = mulmod(x, x, n);
31         d *= 2;
32         if (x == 1) return false;
33         if (x == n - 1) return true;
34     }
35     return false;
36 }
37
38 bool isPrime(ull n) {
39     if (n < 2) return false;
40     if (n < 4) return true;
41     if (n % 2 == 0) return false;
42     ull test_bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
43     for (ull a : test_bases) {
44         if (a >= n) break;
45         if (!millerRabin(n, a)) return false;
46     }
47     return true;
48 }
```

### 4.4 Sieve

```
1  const int N = 1e6 + 5;
2  vector<bool> is_prime(N, true);
3  void sieve(int n) {
4      is_prime[0] = is_prime[1] = false;
5      for (int i = 2; i * i <= n; i++) {
6          if (is_prime[i]) {
7              for (int j = i * i; j <= n; j += i) {
8                  is_prime[j] = false;
9              }
10         }
11     }
12 }
```

### 4.5 SPF

```
1  const int N = 1e6 + 1;
2  vector<int> spf(N);
3  void sieve() {
4      for (int i = 2; i < N; i++) {
5          if (spf[i] == 0) {
6              for (int j = i; j < N; j += i) {
7                  if (spf[j] == 0)
8                      spf[j] = i;
9              }
10         }
11     }
12 }
```

### 4.6 Prime factorizations

```
vector < int > prime_factorizations (ll n) {
    vector < int > factors;
    while(not(n & 1)) factors.push_back(2), n /= 2;
    for(int i = 3; i <= sqrt(n); i += 2)
        while(n % i == 0) factors.push_back(i), n /= i;
    if(n > 2) factors.push_back(n);
    return factors;
}
```

### 4.7 Prime factorizations + SPF

```
vector<int> primeFactors(int x) {
    vector<int> factors;
    while (x != 1) {
        factors.push_back(spf[x]);
        x /= spf[x];
    }
    return factors;
}
```

### 4.8 Prime factorization (Big numbers)

```
typedef unsigned long long ull;
typedef __uint128_t u128;
typedef long long ll;

ull mul(ull a, ull b, ull mod) {
    return ((u128)a * b) % mod;
}

ull power(ull a, ull b, ull mod) {
    ull res = 1;
    a %= mod;
    while (b) {
        if (b & 1) res = mul(res, a, mod);
        a = mul(a, a, mod);
        b >>= 1;
    }
    return res;
}

bool miller_rabin(ull n) {
    if (n < 2 || n % 2 == 0) return n == 2;
    ull d = n - 1, s = 0;
    while (d % 2 == 0) d /= 2, s++;

    ull bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31};
    for (ull a : bases) {
        if (a >= n) break;
        ull x = power(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool ok = false;
        for (int r = 1; r < s; r++) {
            x = mul(x, x, n);
            if (x == n - 1) {
                ok = true;
                break;
```

```
            }
        }
        if (!ok) return false;
    }
    return true;
}

ull pollard_rho(ull n) {
    if (n % 2 == 0) return 2;
    ull c = 1;
    while (true) {
        ull x = 2, y = 2, d = 1;
        auto f = [&](ull x) { return (mul(x, x, n) + c) % n; };
        while (d == 1) {
            x = f(x);
            y = f(f(y));
            d = __gcd((ull)abs((ll)x - (ll)y), n);
        }
        if (d != n) return d;
        c++;
    }
}

void factor(ull n, vector<ull> &factors) {
    if (n == 1) return;
    if (miller_rabin(n)) {
        factors.push_back(n);
        return;
    }
    ull d = pollard_rho(n);
    factor(d, factors);
    factor(n / d, factors);
}
```

### 4.8.1 Usage example

```
int main() {
    vector<ull> factors;
    factor(1e17+9ULL, factors);
    sort(factors.begin(), factors.end());
    for (ull f : factors) cout << f << ' ';
}
```

## 4.9 Segmented Sieve

```
typedef long long ll;
const int MAX = 1e6 + 1;
vector<ll> get_base_primes(int limit) {
    vector<bool> is_prime(limit + 1, true);
    vector<ll> primes;
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i <= limit; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i * 2; j <= limit; j += i)
                is_prime[j] = false;
        }
    }
```

```
    return primes;
}
vector<ll> segmented_sieve(ll L, ll R) {
    int limit = sqrt(R) + 1;
    vector<ll> base = get_base_primes(limit);

    int size = R - L + 1;
    vector<bool> is_prime(size, true);

    for (ll p : base) {
        ll start = max(p * p, (L + p - 1) / p * p);
        for (ll j = start; j <= R; j += p) {
            is_prime[j - L] = false;
        }
    }

    if (L == 1) is_prime[0] = false;

    vector<ll> primes;
    for (int i = 0; i < size; ++i) {
        if (is_prime[i])
            primes.push_back(L + i);
    }
    return primes;
}
```

### 4.10 nCr & nPr

```
ll nCr (ll n, ll r) {
    if(r > n) return 0;
    ll p = 1, k = 1;
    if (n - r < r) r = n - r;
    if(n < 1) return 0;
    while (r > 0){
        p *= n, k *= r;
        ll m = __gcd(p, k);
        p /= m, k /= m, n--, r--;
    }
    return p;
}
ll nPr(ll n, ll r){
    if(r > n) return 0;
    ll npr = 1;
    while(r-- > 0)
        npr *= n--;
    return npr;
}
```

### 4.11 nCr & nPr with Precompute

```
const int N = 2e5+5, MOD = 1e9+7;
ll fact[N], invFact[N];
ll modinv(ll a, ll m) {
    ll res = 1, b = m - 2;
    while (b) {
        if (b & 1) res = res * a % m;
        a = a * a % m;
        b >>= 1;
    }
```

```
    return res;
}
void init_fact() {
    fact[0] = invFact[0] = 1;
    for (int i = 1; i < N; ++i)
        fact[i] = fact[i-1] * i % MOD;
    invFact[N-1] = modinv(fact[N-1], MOD);
    for (int i = N-2; i >= 1; --i)
        invFact[i] = invFact[i+1] * (i+1) % MOD;
}

ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}
ll nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[n - r] % MOD;
}
```

### 4.12 nCr with mod – Lucas Theorem

```
typedef long long ll;
const int MAX_P = 100005;
ll fact[MAX_P], invFact[MAX_P];

ll modpow(ll a, ll b, ll mod) {
    ll res = 1;
    a %= mod;
    while (b) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}

void precompute_fact(ll p) {
    fact[0] = invFact[0] = 1;
    for (ll i = 1; i < p; ++i)
        fact[i] = fact[i - 1] * i % p;
    invFact[p - 1] = modpow(fact[p - 1], p - 2, p);
    for (ll i = p - 2; i >= 1; --i)
        invFact[i] = invFact[i + 1] * (i + 1) % p;
}

// Now O(1) nCr mod p (for small p)
ll small_nCr(ll n, ll r, ll p) {
    if (r > n) return 0;
    return fact[n] * invFact[r] % p * invFact[n - r] % p;
}

// Lucas Theorem (recursive)
ll lucas_nCr(ll n, ll r, ll p) {
    if (r == 0) return 1;
    return lucas_nCr(n / p, r / p, p) * small_nCr(n % p, r % p, p) % p;
}

// Lucas-based nPr using nCr * r!
ll lucas_nPr(ll n, ll r, ll p) {
```

```
    if (r > n) return 0;
    ll comb = lucas_nCr(n, r, p);
    ll r_fact = 1;
    for (ll i = 1; i <= r; ++i)
        r_fact = r_fact * i % p;
    return comb * r_fact % p;
}
```

#### 4.12.1 Usage example

```
int main() {
    ll n = 1e18, r = 1e6;
    ll mod = 9941; // small prime
    precompute_fact(mod); // O(mod)

    // time is O(log base mod for n)
    // consider O(1)
    cout << lucas_nCr(n, r, mod) << "\n";
}
```

### 4.13 Mobius Function (M(n))

```
const int N = 1e6 + 5;
int mobius[N];
// O(n log log n)
void compute_mobius() {
    for (int i = 0; i < N; ++i) mobius[i] = 1;

    vector<int> primes;
    vector<bool> is_prime(N, true);

    for (int i = 2; i < N; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (int j = i; j < N; j += i) {
                is_prime[j] = false;
                mobius[j] *= -1;
            }
            for (ll j = 1LL * i * i; j < N; j += 1LL * i * i) {
                mobius[j] = 0; // square factor
            }
        }
    }
}
```

### 4.14 Matrix Exponentiation

```
const int mod = 1e9 + 7;
ll add(ll a, ll b) {
    return (a + b) % mod;
}
ll mul(ll a, ll b) {
    return a * b % mod;
}
vector<vector<int>> matrix_mul(const vector<vector<int>>& A, const
    vector<vector<int>>& B) {
    int rowsA = A.size();
    int colsA = A[0].size();
```

```
    int colsB = B[0].size();
    vector<vector<int>> result(rowsA, vector<int>(colsB, 0));
    for (int i = 0; i < rowsA; i++) {
        for (int j = 0; j < colsB; j++) {
            for (int k = 0; k < colsA; k++) {
                result[i][j] = add(result[i][j], mul(A[i][k] , B[k][j]));
            }
        }
    }
    return result;
}
vector<vector<int>> matrix_power(vector<vector<int>> base, int exp) {
    int n = base.size();
    vector<vector<int>> result(n, vector<int>(n, 0));
    for (int i = 0; i < n; ++i) result[i][i] = 1;
    while (exp > 0) {
        if (exp & 1) result = matrix_mul(result, base);
        base = matrix_mul(base, base);
        exp >>= 1;
    }
    return result;
}
```

### 4.15 Matrix Exponentiation Many Queries

```
const int mod = 1e9 + 7;
ll add(ll a, ll b) {
    return (a + b) % mod;
}
ll mul(ll a, ll b) {
    return a * b % mod;
}
vector<vector<int>> matrix_mul(const vector<vector<int>>& A, const
   vector<vector<int>>& B) {
    int rowsA = A.size();
    int colsA = A[0].size();
    int colsB = B[0].size();
    vector<vector<int>> result(rowsA, vector<int>(colsB, 0));
    for (int i = 0; i < rowsA; i++) {
        for (int j = 0; j < colsB; j++) {
            for (int k = 0; k < colsA; k++) {
                result[i][j] = add(result[i][j], mul(A[i][k] , B[k][j]));
            }
        }
    }
    return result;
}
struct matrixExpoQueryies {
    private:
    int n, bits;
    vector<vector<vector<int>>>dp;
    void init() {
        for (int i = 1; i < bits; i++) {
            dp[i] = matrix_mul(dp[i-1], dp[i-1]);
        }
    };
    public:
    matrixExpoQueryies(vector<vector<int>>matrix, int _bits = 32) {
        bits = _bits;
        n = matrix.size();
```

```
        dp.resize(bits, vector<vector<int>>(n, vector<int>(n, 0)));
        dp[0] = matrix;
        init();
    }
    vector<int> query_row(int row, int power) { // indexed 0
        vector<int> vec(n, 0);
        vec[row] = 1;
        for (int bit = 0; bit < bits; bit++) {
            if (power & (1 << bit)) {
                vector<int> new_vec(n, 0);
                for (int i_node = 0; i_node < n; i_node++) {
                    if (vec[i_node] == 0) continue;
                    for (int j_node = 0; j_node < n; j_node++) {
                        new_vec[j_node] = (new_vec[j_node] + (ll)vec[i_node]
                            * dp[bit][i_node]
[j_node]) % mod;
                    }
                }
                vec = move(new_vec);
            }
        }
        return vec;
    }
};
```

### 4.16 FFT

```
typedef complex<double> cd;
const double PI = acos(-1);
// Iterative FFT
void fft(vector<cd> &a, bool invert) {
    int n = a.size();
    // Bit-reversal permutation
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }
    // Cooley-Tukey FFT
    for (int len = 2; len <= n; len <<= 1) {
        double angle = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(angle), sin(angle));

        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; ++j) {
                cd u = a[i + j];
                cd v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (cd &x : a)
            x /= n;
```

```
    }
}
// Multiply polynomials
vector<int> multiply(const vector<int> &a, const vector<int> &b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)(a.size() + b.size()))
        n <<= 1;
    fa.resize(n);
    fb.resize(n);
    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; ++i)
        fa[i] *= fb[i];
    fft(fa, true);
    vector<int> result(n);
    for (int i = 0; i < n; ++i)
        result[i] = round(fa[i].real()); // round to nearest integer
    return result;
}
```

### 4.17 NTT with MOD

```
using ll = long long;

const int MOD = 998244353; // Must be prime of the form c*2^k + 1
const int G = 3;           // Primitive root modulo MOD

int modpow(int base, int exp, int mod) {
    int res = 1;
    while (exp > 0) {
        if (exp % 2) res = 1LL * res * base % mod;
        base = 1LL * base * base % mod;
        exp /= 2;
    }
    return res;
}

void ntt(vector<int> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        int wlen = modpow(G, (MOD - 1) / len, MOD);
        if (invert)
            wlen = modpow(wlen, MOD - 2, MOD);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j];
                int v = 1LL * a[i + j + len / 2] * w % MOD;
                a[i + j] = (u + v < MOD ? u + v : u + v - MOD);
                a[i + j + len / 2] = (u - v >= 0 ? u - v : u - v + MOD);
                w = 1LL * w * wlen % MOD;
```

```
            }
        }
    }

    if (invert) {
        int inv_n = modpow(n, MOD - 2, MOD);
        for (int &x : a)
            x = 1LL * x * inv_n % MOD;
    }
}

vector<int> multiply(vector<int> const &a, vector<int> const &b) {
    vector<int> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < (int)(a.size() + b.size()))
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    ntt(fa, false);
    ntt(fb, false);
    for (int i = 0; i < n; ++i)
        fa[i] = 1LL * fa[i] * fb[i] % MOD;
    ntt(fa, true);

    return fa;
}
```

### 4.18 Check Primitive Root

```
int modpow(int a, int b, int mod) {
    int res = 1;
    while (b > 0) {
        if (b & 1) res = 1LL * res * a % mod;
        a = 1LL * a * a % mod;
        b >>= 1;
    }
    return res;
}
// Find prime factors of n
vector<int> factor(int n) {
    vector<int> res;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            res.push_back(i);
            while (n % i == 0)
                n /= i;
        }
    }
    if (n > 1)
        res.push_back(n);
    return res;
}

int find_primitive_root(int mod) {
    int phi = mod - 1;
    vector<int> factors = factor(phi);
    for (int g = 2; g < mod; g++) {
        bool ok = true;
        for (int p : factors) {
```

```
            if (modpow(g, phi / p, mod) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return g;
    }
    return -1; // Should never happen if mod is prime
}

int main() {
    int mod = 998244353;
    int root = find_primitive_root(mod);
    cout << "Primitive root of " << mod << " is " << root << '\n';
}
```

### 4.19 Solve equation

```
pair<ll,ll> equ(ll a,ll b,ll c)
{
    ll x1 = (-b + sqrt(b * b - 4 * a * c)) / (2 *a);
    ll x2 = (-b - sqrt(b * b - 4 * a * c)) / (2 * a);
    return {x1,x2};
}
```

### 4.20 Gaussian elimination

```
void gaussian_elimination(vector<vector<double>>& mat, int n) {
    for (int i = 0; i < n; i++) {
        // Partial pivoting
        int maxRow = i;
        for (int k = i + 1; k < n; k++)
            if (abs(mat[k][i]) > abs(mat[maxRow][i]))
                maxRow = k;

        swap(mat[i], mat[maxRow]);

        // Make leading coefficient = 1
        double lead = mat[i][i];
        for (int j = i; j <= n; j++)
            mat[i][j] /= lead;

        // Eliminate all rows below and above
        for (int k = 0; k < n; k++) {
            if (k != i) {
                double factor = mat[k][i];
                for (int j = i; j <= n; j++)
                    mat[k][j] -= factor * mat[i][j];
            }
        }
    }

    // Print result
    cout << fixed << setprecision(4);
    for (int i = 0; i < n; i++)
        cout << "Variable " << i + 1 << " = " << mat[i][n] << '\n';
}
```

### 4.20.1 Usage example

```cpp
int main() {
    int n;
    cout << "Enter number of variables: ";
    cin >> n;
    vector<vector<double>> mat(n, vector<double>(n + 1));
    cout << "Enter coefficients and constants for each equation:\n";
    for (int i = 0; i < n; i++) {
        cout << "Equation " << i + 1 << ":\n";
        for (int j = 0; j <= n; j++) {
            cin >> mat[i][j];
        }
    }
    gaussian_elimination(mat, n);
    return 0;
}
```

# 5 Geometry

## 5.1 define some struct

```cpp
const double EPS = 1e-9;
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}

    Point operator+(const Point& p) const { return {x + p.x, y + p.y}; }
    Point operator-(const Point& p) const { return {x - p.x, y - p.y}; }
    Point operator*(double d) const { return {x * d, y * d}; }
    Point operator/(double d) const { return {x / d, y / d}; }

    double dot(const Point& p) const { return x * p.x + y * p.y; }
    double cross(const Point& p) const { return x * p.y - y * p.x; }
    double dist2() const { return x * x + y * y; }
    double dist() const { return sqrt(dist2()); }

    bool operator==(const Point& p) const {
        return fabs(x - p.x) < EPS && fabs(y - p.y) < EPS;
    }
};

struct Line {
    double A, B, C;
    Line(Point p1, Point p2) {
        A = p2.y - p1.y;
        B = p1.x - p2.x;
        C = -(A * p1.x + B * p1.y);
    }
    Line(double A, double B, double C) : A(A), B(B), C(C) {}
};
struct Circle {
    Point center;
    double r;
    Circle(Point c, double radius) : center(c), r(radius) {}
};
```

## 5.2 relation between two lines

```cpp
bool parallel(Line l1, Line l2) {
    return fabs(l1.A * l2.B - l2.A * l1.B) < EPS;
}
bool sameLine(Line l1, Line l2) {
    return parallel(l1, l2) && fabs(l1.A * l2.C - l2.A * l1.C) < EPS &&
       fabs(l1.B * l2.C - l2.B * l1.C) < EPS;
}
bool intersect(Line l1, Line l2, Point& p) {
    double det = l1.A * l2.B - l2.A * l1.B;
    if (fabs(det) < EPS) return false;  // Parallel or same line
    p.x = (l2.C * l1.B - l1.C * l2.B) / det;
    p.y = (l1.C * l2.A - l2.C * l1.A) / det;
    return true;
}
```

### 5.3 distance from point to (line — segment)

```cpp
double distanceToLine(Point p, Line l) {
    return fabs(l.A * p.x + l.B * p.y + l.C) / hypot(l.A, l.B);
}

// Distance from point p to line segment ab
double distanceToSegment(Point p, Point a, Point b) {
    if (a == b) return (p - a).dist();
    Point ap = p - a, ab = b - a;
    double u = ap.dot(ab) / ab.dot(ab);
    u = max(0.0, min(1.0, u));
    Point proj = a + ab * u;
    return (p - proj).dist();
}
```

### 5.4 Check if point on segment

```cpp
// Check if point p lies on segment ab
bool onSegment(Point p, Point a, Point b) {
    return fabs((b - a).cross(p - a)) < EPS && (p - a).dot(p - b) <= 0;
}
```

### 5.5 Check if two segments intersect

```cpp
// Check if segments ab and cd intersect
bool segmentsIntersect(Point a, Point b, Point c, Point d) {
    double d1 = (d - c).cross(a - c);
    double d2 = (d - c).cross(b - c);
    double d3 = (b - a).cross(c - a);
    double d4 = (b - a).cross(d - a);

    if ((d1 * d2 < 0) && (d3 * d4 < 0)) return true;
    if (fabs(d1) < EPS && onSegment(a, c, d)) return true;
    if (fabs(d2) < EPS && onSegment(b, c, d)) return true;
    if (fabs(d3) < EPS && onSegment(c, a, b)) return true;
    if (fabs(d4) < EPS && onSegment(d, a, b)) return true;
    return false;
}
```

## 5.6 Intersection between two lines

```
bool lineLineIntersection(Point a, Point b, Point c, Point d, Point& inter) {
    Point ab = b - a, cd = d - c;
    double det = ab.cross(cd);
    if (fabs(det) < EPS) return false;  // Parallel
    double t = (c - a).cross(cd) / det;
    inter = a + ab * t;
    return true;
}
```

## 5.7 Angle between two lines

```
double angleBetween(Point a, Point b, Point c, Point d) {
    Point v1 = b - a, v2 = d - c;
    double dot_prod = v1.dot(v2);
    double len = v1.dist() * v2.dist();
    return acos(dot_prod / len);
}
```

## 5.8 Find points at dist a from p1 and dist b from p2

```
// Find points at distance a from p1 and distance b from p2
vector<Point> findIntersection(Point p1, Point p2, double a, double b) {
    double dx = p2.x - p1.x;
    double dy = p2.y - p1.y;
    double d = hypot(dx, dy);
    vector<Point> result;

    // No solution: too far, too close, or infinite
    if (d > a + b || d < fabs(a - b) || (d == 0 && a == b))
        return result;

    double x = (a * a - b * b + d * d) / (2 * d);
    double h = sqrt(max(0.0, a * a - x * x));

    double xm = p1.x + x * dx / d;
    double ym = p1.y + x * dy / d;

    double rx = -dy * (h / d);
    double ry = dx * (h / d);

    result.push_back({xm + rx, ym + ry});
    if (h > 0)
        result.push_back({xm - rx, ym - ry});

    return result;
}
```

## 5.9 Triangle area

### 5.9.1 Triangle area from points

```
double areaTriangle(Point a, Point b, Point c) {
    return abs((b - a).cross(c - a)) / 2.0;
}
```

### 5.9.2 Triangle area from sides

```
double areaHeron(double a, double b, double c) {
    double s = (a + b + c) / 2.0;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}
```

## 5.10 Classify triangle & Angle type

```
string classifyTriangle(double a, double b, double c) {
    if (a == b && b == c) return "Equilateral";
    if (a == b || b == c || a == c) return "Isosceles";
    return "Scalene";
}
string angleType(double a, double b, double c) {
    vector<double> sides = {a, b, c};
    sort(sides.begin(), sides.end());
    double x = sides[0], y = sides[1], z = sides[2];
    if (x * x + y * y == z * z) return "Right";
    if (x * x + y * y < z * z) return "Obtuse";
    return "Acute";
}
```

## 5.11 Centroid of triangle

```
Point centroid(Point a, Point b, Point c) {
    return Point((a.x + b.x + c.x) / 3.0, (a.y + b.y + c.y) / 3.0);
}
```

## 5.12 Circle inside triangle

```
Point incenter(Point a, Point b, Point c) {
    double ab = distance(b, c);
    double bc = distance(c, a);
    double ca = distance(a, b);
    double perimeter = ab + bc + ca;
    return Point((a.x * ab + b.x * bc + c.x * ca) / perimeter,
                 (a.y * ab + b.y * bc + c.y * ca) / perimeter);
}
double inradius(double a, double b, double c) {
    double s = (a + b + c) / 2.0;
    return sqrt((s - a) * (s - b) * (s - c) / s);
}
```

## 5.13 Circle outside triangle

```
Point circumcenter(Point A, Point B, Point C) {
    double D = 2 * (A.x * (B.y - C.y) +
                    B.x * (C.y - A.y) +
                    C.x * (A.y - B.y));
    double Ux = ((A.x * A.x + A.y * A.y) * (B.y - C.y) +
                 (B.x * B.x + B.y * B.y) * (C.y - A.y) +
                 (C.x * C.x + C.y * C.y) * (A.y - B.y)) /
                D;
    double Uy = ((A.x * A.x + A.y * A.y) * (C.x - B.x) +
                 (B.x * B.x + B.y * B.y) * (A.x - C.x) +
```

```
                  (C.x * C.x + C.y * C.y) * (B.x - A.x)) /
                 D;
    return {Ux, Uy};
}
double circumradius(double a, double b, double c) {
    double area = areaHeron(a, b, c);
    return (a * b * c) / (4.0 * area);
}
```

### 5.14 Check if point inside triangle

```
double angle(double a, double b, double c) {
    return acos((b * b + c * c - a * a) / (2 * b * c));
}
bool pointInTriangle(Point a, Point b, Point c, Point p) {
    double A = areaTriangle(a, b, c);
    double A1 = areaTriangle(p, b, c);
    double A2 = areaTriangle(a, p, c);
    double A3 = areaTriangle(a, b, p);
    return abs(A - (A1 + A2 + A3)) < 1e-9;
}
```

### 5.15 Point to circle relation

```
int pointCircleRelation(Circle c, Point p) {
    double d2 = (p - c.center).dist2();
    double r2 = c.r * c.r;
    if (abs(d2 - r2) < 1e-9) return 0;  // on the circle
    if (d2 < r2) return -1;             // inside the circle
    return 1;                           // outside the circle
}
```

### 5.16 Circle to Circle relation

```
int circleCircleRelation(Circle c1, Circle c2) {
    double d = (c1.center - c2.center).dist();
    if (d > c1.r + c2.r + 1e-9) return 0;
                                                    // no intersection
       (outside)
    if (d + min(c1.r, c2.r) < max(c1.r, c2.r) - 1e-9) return 0;
                              // one inside the other
    if (abs(d - (c1.r + c2.r)) < 1e-9 || abs(d - abs(c1.r - c2.r)) < 1e-9)
       return 1;  // touch
    return 2;

       // intersect at two points
}
```

### 5.17 intersection between line & circle

```
vector<Point> circleLineIntersection(Circle c, Point a, Point b) {
    Point d = b - a;
    Point f = a - c.center;

    double A = d.dot(d);
    double B = 2 * f.dot(d);
    double C = f.dot(f) - c.r * c.r;
```

```
    double D = B * B - 4 * A * C;
    vector<Point> res;

    if (D < -1e-9) return res;  // no intersection
    D = max(D, 0.0);            // numerical fix

    double t1 = (-B + sqrt(D)) / (2 * A);
    double t2 = (-B - sqrt(D)) / (2 * A);

    res.push_back(a + d * t1);
    if (D > 1e-9) res.push_back(a + d * t2);  // two distinct points

    return res;
}
```

### 5.18 Tangents from point to circle

```
vector<Point> tangentsFromPointToCircle(Point p, Circle c) {
    vector<Point> res;
    double d2 = (p - c.center).dist2();
    double r2 = c.r * c.r;
    if (d2 < r2 - 1e-9) return res;  // no tangent
    if (abs(d2 - r2) < 1e-9) {
        // exactly one tangent (point on circle)
        Point dir = p - c.center;
        Point unit = Point(-dir.y, dir.x) * (c.r / dir.dist());
        res.push_back(p + unit);
        return res;
    }
    double d = sqrt(d2);
    double l = sqrt(d2 - r2);
    Point dir = (p - c.center) * (c.r / d);
    Point orth = Point(-dir.y, dir.x) * (l / c.r);

    res.push_back(c.center + dir + orth);
    res.push_back(c.center + dir - orth);
    return res;
}
```

### 5.19 intersection area between two circles

```
double circle_intersection_area(const Circle& c1, const Circle& c2) {
    double d = (c1.center - c2.center).dist();

    // No overlap
    if (d >= c1.r + c2.r - EPS)
        return 0.0;

    // One circle completely inside the other
    if (d <= fabs(c1.r - c2.r) + EPS) {
        double r = min(c1.r, c2.r);
        return M_PI * r * r;
    }

    double r1 = c1.r, r2 = c2.r;

    double alpha = acos((d * d + r1 * r1 - r2 * r2) / (2 * d * r1)) * 2;
    double beta = acos((d * d + r2 * r2 - r1 * r1) / (2 * d * r2)) * 2;
```

```
18
19      double area1 = 0.5 * r1 * r1 * (alpha - sin(alpha));
20      double area2 = 0.5 * r2 * r2 * (beta - sin(beta));
21
22      return area1 + area2;
23  }
```

### 5.20 Polygon area

```
1  double polygon_area(const vector<Point>& points) {
2      int n = points.size();
3      double area = 0;
4      for (int i = 0; i < n; i++) {
5          area += points[i].cross(points[(i + 1) % n]);
6      }
7      return fabs(area) / 2.0;
8  }
```

### 5.21 Polygon perimeter

```
1  double polygon_perimeter(const vector<Point>& pts) {
2      int n = pts.size();
3      double len = 0;
4      for (int i = 0; i < n; i++) {
5          len += (pts[i] - pts[(i + 1) % n]).dist();
6      }
7      return len;
8  }
```

### 5.22 Check Polygon is convex

```
1  bool is_convex(const vector<Point>& pts) {
2      int n = pts.size(), sign = 0;
3      for (int i = 0; i < n; i++) {
4          Point a = pts[i];
5          Point b = pts[(i + 1) % n];
6          Point c = pts[(i + 2) % n];
7
8          double cross = (b - a).cross(c - b);
9          if (fabs(cross) > 1e-9) {
10             int current_sign = cross > 0 ? 1 : -1;
11             if (sign == 0)
12                 sign = current_sign;
13             else if (sign != current_sign)
14                 return false;
15         }
16     }
17     return true;
18 }
```

### 5.23 Check point in polygon

```
1  bool point_in_polygon(const vector<Point>& poly, const Point& p) {
2      int cnt = 0, n = poly.size();
3      for (int i = 0; i < n; i++) {
4          Point a = poly[i];
5          Point b = poly[(i + 1) % n];
```

```
        if (a.y > b.y) swap(a, b);

        if (fabs((p - a).cross(b - a)) < 1e-9 &&
            (p - a).dot(p - b) <= 1e-9) return true; // on edge

        if (p.y > a.y && p.y <= b.y && (b - a).cross(p - a) > 0)
            cnt ^= 1;
    }
    return cnt;
}
```

### 5.24 Clip convex polygon

```
// Sutherland Hodgman Polygon Clipping (for convex clip polygon)
vector<Point> polygon_clip(const vector<Point>& subject, const
    vector<Point>& clip) {
    auto inside = [](Point p, Point a, Point b) {
        return (b - a).cross(p - a) >= -EPS;
    };

    auto intersection = [](Point a, Point b, Point c, Point d) {
        Point r = b - a, s = d - c;
        double t = (c - a).cross(s) / r.cross(s);
        return a + r * t;
    };

    vector<Point> output = subject;
    int n = clip.size();
    for (int i = 0; i < n; i++) {
        Point a = clip[i], b = clip[(i + 1) % n];
        vector<Point> input = output;
        output.clear();
        for (int j = 0; j < input.size(); j++) {
            Point p = input[j], q = input[(j + 1) % input.size()];
            bool pin = inside(p, a, b);
            bool qin = inside(q, a, b);
            if (pin && qin) output.push_back(q);
            else if (pin && !qin) output.push_back(intersection(p, q, a, b));
            else if (!pin && qin) {
                output.push_back(intersection(p, q, a, b));
                output.push_back(q);
            }
        }
    }
    return output;
}
```

### 5.25 Convex Hull

```
vector<Point> convex_hull(vector<Point> pts) {
    sort(pts.begin(), pts.end());
    int n = pts.size(), k = 0;
    vector<Point> hull(2 * n);

    auto cross = [](const Point& o, const Point& a, const Point& b) {
        return (a - o).cross(b - o);
    };

    // Lower hull
```

```
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(hull[k - 2], hull[k - 1], pts[i]) <= 0) k--;
        hull[k++] = pts[i];
    }
    // Upper hull
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(hull[k - 2], hull[k - 1], pts[i]) <= 0) k--;
        hull[k++] = pts[i];
    }

    hull.resize(k - 1);
    return hull;
}
```

# 6 Big Numbers

## 6.1 add

```
string add(string a, string b) {
    if (a.length() < b.length()) swap(a, b);

    int carry = 0;
    string result = "";

    int diff = a.length() - b.length();
    for (int i = b.length() - 1; i >= 0; i--) {
        int sum = (a[i + diff] - '0') + (b[i] - '0') + carry;
        carry = sum / 10;
        result += (sum % 10 + '0');
    }

    for (int i = diff - 1; i >= 0; i--) {
        int sum = (a[i] - '0') + carry;
        carry = sum / 10;
        result += (sum % 10 + '0');
    }

    if (carry) result += (carry + '0');
    reverse(result.begin(), result.end());
    return result;
}
```

## 6.2 subtract

```
string subtract(string a, string b) {
    // Make sure a >= b
    if (a.length() < b.length()) swap(a, b);
    else if (a.length() == b.length() && a < b) swap(a, b);

    // Pad b with leading zeros
    while (b.length() < a.length())
        b = '0' + b;

    string result = "";
    int carry = 0;

    for (int i = a.length() - 1; i >= 0; i--) {
        int digitA = a[i] - '0';
```

```
        int digitB = b[i] - '0' + carry;

        if (digitA < digitB) {
            digitA += 10;
            carry = 1;
        } else {
            carry = 0;
        }

        result += (digitA - digitB) + '0';
    }

    // Remove leading zeros
    while (result.size() > 1 && result.back() == '0')
        result.pop_back();

    reverse(result.begin(), result.end());
    return result;
}
```

### 6.3 compare

```
int compare(string a, string b) {
    if (a.length() > b.length()) return 1;
    if (a.length() < b.length()) return -1;
    return a.compare(b);
}
```

### 6.4 multiply with int

```
string multiply_by_int(string num, int multiplier) {
    if (multiplier == 0 || num == "0") return "0";

    string result = "";
    int carry = 0;

    for (int i = num.size() - 1; i >= 0; --i) {
        int product = (num[i] - '0') * multiplier + carry;
        carry = product / 10;
        result += (product % 10) + '0';
    }

    while (carry > 0) {
        result += (carry % 10) + '0';
        carry /= 10;
    }

    reverse(result.begin(), result.end());
    return result;
}
```

### 6.5 divide by int

```
pair<string, int> divide_by_int(string num, int divisor) {
    if (divisor == 0) {
        throw invalid_argument("Division by zero");
    }

```

```
    string quotient = "";
    int remainder = 0;

    for (char digit : num) {
        int current = remainder * 10 + (digit - '0');
        quotient += (current / divisor) + '0';
        remainder = current % divisor;
    }

    // Remove leading zeros
    size_t first_non_zero = quotient.find_first_not_of('0');
    if (first_non_zero != string::npos)
        quotient = quotient.substr(first_non_zero);
    else
        quotient = "0";

    return {quotient, remainder}; // Quotient and Remainder
}
```

### 6.6 int prime MOD

```
ll Big_Mod(string s, ll mod){
    ll res = 0;
    for(auto& c : s)
        res = (res * 10 + (c - '0')) % mod;
    return res;
}
```

### 6.7 Mod with big number

```
// Big integer modulo: a % b
// O(n*m)
string mod(string a, string b) {
    string remainder = "";

    for (char digit : a) {
        remainder += digit;

        // Remove leading zeros
        while (remainder.size() > 1 && remainder[0] == '0')
            remainder.erase(remainder.begin());

        // Subtract b from remainder while remainder >= b
        while (compare(remainder, b) >= 0) {
            remainder = subtract(remainder, b);
        }
    }

    return remainder;
}
```

## 7 Applications

### 7.1 Binary Trie with XOR

```
struct Node {
    Node* child[2];
```

```
    int count;  // number of times this path is used

    Node() {
        child[0] = child[1] = nullptr;
        count = 0;
    }
};

class BinaryTrie {
private:
    Node* root;
    static const int MAX_BIT = 61;  // for 64-bit integers (0 to 62)

public:
    BinaryTrie() {
        root = new Node();
    }

    // Insert number into the trie
    void insert(ll num) {
        Node* node = root;
        for (int bit = MAX_BIT; bit >= 0; --bit) {
            int b = (num >> bit) & 1;
            if (!node->child[b])
                node->child[b] = new Node();
            node = node->child[b];
            node->count++;
        }
    }

    // Remove one instance of number from the trie
    void erase(ll num) {
        if (!exists(num)) return;  // Safeguard
        Node* node = root;
        for (int bit = MAX_BIT; bit >= 0; --bit) {
            int b = (num >> bit) & 1;
            node = node->child[b];
            node->count--;
        }
    }

    // Check if number exists in trie
    bool exists(ll num) {
        return true;
        Node* node = root;
        for (int bit = MAX_BIT; bit >= 0; --bit) {
            int b = (num >> bit) & 1;
            if (!node->child[b] || node->child[b]->count == 0)
                return false;
            node = node->child[b];
        }
        return true;
    }

    // Find max XOR of num with any element in the trie
    ll maxXOR(ll num) {
        Node* node = root;
        ll res = 0;
        for (int bit = MAX_BIT; bit >= 0; --bit) {
            int b = (num >> bit) & 1;
            ll desired = b ^ 1;
```

```
            if (node->child[desired] && node->child[desired]->count > 0) {
                res |= (1LL << bit);
                node = node->child[desired];
            } else if (node->child[b] && node->child[b]->count > 0) {
                node = node->child[b];
            } else {
                break;  // No further paths
            }
        }
        return res;
    }

    // Find min XOR of num with any element in the trie
    ll minXOR(ll num) {
        Node* node = root;
        ll res = 0;
        for (int bit = MAX_BIT; bit >= 0; --bit) {
            int b = (num >> bit) & 1;
            if (node->child[b] && node->child[b]->count > 0) {
                node = node->child[b];
            } else if (node->child[b^1] && node->child[b^1]->count > 0) {
                res |= (1LL << bit);
                node = node->child[b^1];
            } else {
                break;  // No further paths
            }
        }
        return res;
    }
};
```

### 7.2 Longest increasing subsequence

```
vector<int>LIS(vector<int>&arr) {
    int N = arr.size();
    vector<int> LIS(N, 1);
    vector<int> dp;

    for (int i = 0; i < N; i++) {
        auto it = lower_bound(dp.begin(), dp.end(), arr[i]);
        int pos = distance(dp.begin(), it);
        if (it == dp.end()) dp.push_back(arr[i]);
        else *it = arr[i];
        LIS[i] = pos + 1;
    }
    return LIS;
}
```

### 7.3 Segment Tree with Hashing

```
const int N = 2e5 + 1;
const int M = 1e9 + 7;
const int P = 31;
int pw[N];

int add(int a, int b) {
    return (1ll * a + b) % M;
}

```

```
int mul(int a, int b) {
    return (1ll * a * b) % M;
}

int sub(int a, int b) {
    return (1ll * a - b + M) % M;
}

int seg[N * 4], lazy[4 * N], nums[N], n, m, k;
int merge(int left, int right, int rsize) {
    return add(right, mul(left, pw[rsize]));
}

int getHash(int d, int len) {
    if (len == 0) return 0;

    int ret = getHash(d, len >> 1);
    ret = merge(ret, ret, len >> 1);
    if (len & 1) ret = merge(ret, d, 1);

    return ret;
}

void push(int i, int low, int high) {
    if (lazy[i] == -1) return;
    if (low < high) {
        lazy[2 * i] = lazy[i];
        lazy[2 * i + 1] = lazy[i];
    }
    seg[i] = getHash(lazy[i], high - low + 1);
    lazy[i] = -1;
}

void build(int i, int low, int high) {
    if (low == high) {
        seg[i] = nums[low];
        return;
    }

    int mid = low + high >> 1;
    build(2 * i, low, mid);
    build(2 * i + 1, mid + 1, high);
    seg[i] = merge(seg[2 * i], seg[2 * i + 1], high - mid);
}

pair<int, int> get(int i, int low, int high, int l, int r) {
    push(i, low, high);
    if (high < l || r < low) return {0, 0};
    if (l <= low && r >= high) return {seg[i], high - low + 1};
    int mid = low + high >> 1;

    pair<int, int> left = get(2 * i, low, mid, l, r);
    pair<int, int> right = get(2 * i + 1, mid + 1, high, l, r);

    return {merge(left.first, right.first, right.second), left.second +
        right.second};
}

int get(int l, int r) {
    return get(1, 1, n, l, r).first;
}
```

```

void update(int i, int low, int high, int l, int r, int d) {
    push(i, low, high);
    if (high < l || r < low) return;
    if (l <= low && r >= high) {
        lazy[i] = d;
        push(i, low, high);
        return;
    }

    int mid = low + high >> 1;
    update(2 * i, low, mid, l, r, d);
    update(2 * i + 1, mid + 1, high, l, r, d);
    seg[i] = merge(seg[2 * i], seg[2 * i + 1], high - mid);
}

void update(int l, int r, int d) {
    return update(1, 1, n, l, r, d);
}
```

## 7.4 Distinct Elements in range

```
vector<int> build_index(const vector<int>& input) {
    int n = input.size();
    unordered_map<int, int> last;
    vector<int> next(n, n);

    for (int i = n - 1; i >= 0; --i) {
        if (last.find(input[i]) != last.end()) {
            next[i] = last[input[i]];
        }
        last[input[i]] = i;
    }

    return next;
}

vector<int> merge(const vector<int>& left, const vector<int>& right) {
    vector<int> result;
    size_t p = 0, q = 0;

    while (p < left.size() && q < right.size()) {
        if (left[p] < right[q]) {
            result.push_back(left[p++]);
        } else {
            result.push_back(right[q++]);
        }
    }

    while (p < left.size()) {
        result.push_back(left[p++]);
    }

    while (q < right.size()) {
        result.push_back(right[q++]);
    }

    return result;
}

```

```cpp
struct SegmentTree {
    vector<int> merged;
    SegmentTree* left = nullptr;
    SegmentTree* right = nullptr;

    SegmentTree(const vector<int>& input, int l, int r) {
        if (r - l == 1) {
            merged = { input[l] };
        } else {
            int mid = (l + r) / 2;
            left = new SegmentTree(input, l, mid);
            right = new SegmentTree(input, mid, r);
            merged = merge(left->merged, right->merged);
        }
    }

    ~SegmentTree() {
        delete left;
        delete right;
    }

    int query(int tree_left, int tree_right, int query_l, int query_r) {
        if (tree_right - tree_left == 1) {
            return merged[0] >= query_r ? 1 : 0;
        }

        int mid = (tree_left + tree_right) / 2;
        if (query_r <= mid) {
            return left->query(tree_left, mid, query_l, query_r);
        } else if (query_l >= mid) {
            return right->query(mid, tree_right, query_l, query_r);
        } else if (query_l <= tree_left && query_r >= tree_right) {
            return merged.size() - (lower_bound(merged.begin(),
                merged.end(), query_r) - merged.begin());
        } else {
            return left->query(tree_left, mid, query_l, query_r) +
                   right->query(mid, tree_right, query_l, query_r);
        }
    }
};
```

### 7.4.1 Usage Example

```cpp
int main() {
    // query is [L, R [ if size of array 5 the full range is from 0, 5
    vector<int> arr{1, 2, 5, 1, 2, 7};
    vector<int> input = build_index(arr);
    SegmentTree tree(input, 0, arr.size());
    cout << tree.query(0, arr.size(), 0, arr.size());
}
```

## 7.5 Number of subarrays in intervals with (add & remove) interval

```cpp
set<pair<int, int>> ones;
ll total = 0;

ll count_subarrays(int len) {
    return 1LL * len * (len + 1) / 2;
}
```

```cpp

void add_interval(int l, int r) {
    auto it = ones.lower_bound({l, -1});
    if (it != ones.begin()) --it;
    int new_l = l, new_r = r;
    vector<pair<int, int>> to_erase;
    while (it != ones.end() && it->first <= r) {
        int il = it->first, ir = it->second;
        if (ir < l) { ++it; continue; }
        new_l = min(new_l, il);
        new_r = max(new_r, ir);
        total -= count_subarrays(ir - il + 1);
        to_erase.push_back(*it);
        ++it;
    }
    for (auto &seg : to_erase) ones.erase(seg);
    ones.insert({new_l, new_r});
    total += count_subarrays(new_r - new_l + 1);
}

void remove_interval(int l, int r) {
    auto it = ones.lower_bound({l, -1});
    if (it != ones.begin()) --it;
    vector<pair<int, int>> to_erase;
    vector<pair<int, int>> to_add;
    while (it != ones.end() && it->first <= r) {
        int il = it->first, ir = it->second;
        if (ir < l) { ++it; continue; }

        total -= count_subarrays(ir - il + 1);
        to_erase.push_back(*it);

        if (il < l) {
            to_add.emplace_back(il, l - 1);
            total += count_subarrays(l - il);
        }
        if (ir > r) {
            to_add.emplace_back(r + 1, ir);
            total += count_subarrays(ir - r);
        }
        ++it;
    }
    for (auto &seg : to_erase) ones.erase(seg);
    for (auto &seg : to_add) ones.insert(seg);
}
```

### 7.6 Get the L, R of the longest palindrome for many queries

```cpp
struct NodeVal {
    int len;
    int start;
    int end;
    NodeVal() : len(0), start(-1), end(-1) {}
    NodeVal(int l, int s, int e) : len(l), start(s), end(e) {}
};

class SegmentTree {
private:
    vector<NodeVal> tree;
    int n;
```

```
13
14      NodeVal combine(NodeVal a, NodeVal b) {
15          if (a.len >= b.len) {
16              return a;
17          }
18          return b;
19      }
20
21      void update(int idx, NodeVal val, int node, int l, int r) {
22          if (l == r) {
23              if (val.len > tree[node].len) {
24                  tree[node] = val;
25              }
26              return;
27          }
28          int mid = (l + r) / 2;
29          if (idx <= mid) {
30              update(idx, val, 2*node+1, l, mid);
31          } else {
32              update(idx, val, 2*node+2, mid+1, r);
33          }
34          tree[node] = combine(tree[2*node+1], tree[2*node+2]);
35      }
36
37      NodeVal query(int ql, int qr, int node, int l, int r) {
38          if (ql > r || qr < l) return NodeVal(0, -1, -1);
39          if (ql <= l && r <= qr) {
40              return tree[node];
41          }
42          int mid = (l + r) / 2;
43          NodeVal left_val = query(ql, qr, 2*node+1, l, mid);
44          NodeVal right_val = query(ql, qr, 2*node+2, mid+1, r);
45          return combine(left_val, right_val);
46      }
47
48  public:
49      SegmentTree(int size) {
50          n = size;
51          tree.assign(4 * n, NodeVal(0, -1, -1));
52      }
53
54      void update(int idx, NodeVal val) {
55          if (idx < 0 || idx >= n) return;
56          update(idx, val, 0, 0, n-1);
57      }
58
59      NodeVal query(int ql, int qr) {
60          if (ql > qr) return NodeVal(0, -1, -1);
61          return query(ql, qr, 0, 0, n-1);
62      }
63  };
64
65  int main() {
66      string s;
67      cin >> s;
68      int n = s.size();
69      vector<int> d1(n, 0), d2(n, 0);
70
71      for (int i = 0, l = 0, r = -1; i < n; i++) {
72          int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
73          while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) {
```

```
            k++;
        }
        d1[i] = k;
        if (i + k - 1 > r) {
            l = i - k + 1;
            r = i + k - 1;
        }
    }

    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) {
            k++;
        }
        d2[i] = k;
        if (i + k - 1 > r) {
            l = i - k;
            r = i + k - 1;
        }
    }

    vector<tuple<int, int, int>> segs;
    for (int i = 0; i < n; i++) {
        if (d1[i] >= 1) {
            int l1 = i - d1[i] + 1;
            int r1 = i + d1[i] - 1;
            int len1 = 2 * d1[i] - 1;
            segs.push_back({r1, l1, len1});
        }
        if (d2[i] >= 1) {
            int l2 = i - d2[i];
            int r2 = i + d2[i] - 1;
            int len2 = 2 * d2[i];
            segs.push_back({r2, l2, len2});
        }
    }

    sort(segs.begin(), segs.end());

    int q;
    cin >> q;
    vector<tuple<int, int, int>> queries;
    for (int i = 0; i < q; i++) {
        int L, R;
        cin >> L >> R;
        L--; R--;
        queries.push_back({R, L, i});
    }
    sort(queries.begin(), queries.end());

    vector<pair<int, int>> ans_segment(q);
    SegmentTree seg_tree(n);

    int j = 0;
    int q_ptr = 0;
    for (int R = 0; R < n; R++) {
        while (j < segs.size() && get<0>(segs[j]) <= R) {
            int r_end = get<0>(segs[j]);
            int l_start = get<1>(segs[j]);
            int len_val = get<2>(segs[j]);
            NodeVal node_val(len_val, l_start, r_end);
```

```
            seg_tree.update(l_start, node_val);
            j++;
        }
        while (q_ptr < queries.size() && get<0>(queries[q_ptr]) == R) {
            int Lq = get<1>(queries[q_ptr]);
            int idx = get<2>(queries[q_ptr]);
            NodeVal res = seg_tree.query(Lq, R);
            ans_segment[idx] = {res.start, res.end};
            q_ptr++;
        }
    }

    for (int i = 0; i < q; i++) {
        cout << ans_segment[i].first+1 << " " << ans_segment[i].second+1 <<
            '\n';
    }

    return 0;
}
```

### 7.7 Most frequent element in L, R in array

```
const int MAXN = 1e5 + 5;
int a[MAXN], compressed[MAXN], n, q;
vector<int> vals;

map<int, int> val_to_idx;
map<int, int> idx_to_val;

struct Node {
    Node *left, *right;
    int freq, val;
    Node(int f = 0, int v = 0) : left(nullptr), right(nullptr), freq(f),
        val(v) {}
};

Node* update(Node* prev, int l, int r, int pos) {
    if (pos < l || pos > r) return prev;
    Node* cur = new Node();
    if (l == r) {
        cur->freq = (prev ? prev->freq : 0) + 1;
        cur->val = l;
        return cur;
    }
    int mid = (l + r) / 2;
    cur->left = update(prev ? prev->left : nullptr, l, mid, pos);
    cur->right = update(prev ? prev->right : nullptr, mid + 1, r, pos);
    if (cur->left && cur->right) {
        if (cur->left->freq >= cur->right->freq) {
            cur->freq = cur->left->freq;
            cur->val = cur->left->val;
        } else {
            cur->freq = cur->right->freq;
            cur->val = cur->right->val;
        }
    } else {
        cur->freq = cur->left ? cur->left->freq : cur->right->freq;
        cur->val = cur->left ? cur->left->val : cur->right->val;
    }
    return cur;
```

```
}

pair<int, int> query(Node* rNode, Node* lNode, int l, int r) {
    if (l == r) {
        int freq = (rNode ? rNode->freq : 0) - (lNode ? lNode->freq : 0);
        return {freq, l};
    }
    int mid = (l + r) / 2;

    int leftR = rNode && rNode->left ? rNode->left->freq : 0;
    int leftL = lNode && lNode->left ? lNode->left->freq : 0;

    int rightR = rNode && rNode->right ? rNode->right->freq : 0;
    int rightL = lNode && lNode->right ? lNode->right->freq : 0;

    if (leftR - leftL >= rightR - rightL)
        return query(rNode ? rNode->left : nullptr, lNode ? lNode->left :
            nullptr, l, mid);
    else
        return query(rNode ? rNode->right : nullptr, lNode ? lNode->right :
            nullptr, mid+1, r);
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);

    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        vals.push_back(a[i]);
    }

    // Coordinate compression
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (int i = 0; i < vals.size(); ++i) {
        val_to_idx[vals[i]] = i + 1;
        idx_to_val[i + 1] = vals[i];
    }

    int maxVal = vals.size(); // compressed domain size
    vector<Node*> roots(n + 1);
    roots[0] = new Node();

    for (int i = 1; i <= n; i++) {
        compressed[i] = val_to_idx[a[i]];
        roots[i] = update(roots[i - 1], 1, maxVal, compressed[i]);
    }

    while (q--) {
        int l, r; cin >> l >> r;
        auto [freq, comp_val] = query(roots[r], roots[l - 1], 1, maxVal);
        cout << idx_to_val[comp_val] << " appears " << freq << " times\n";
    }

    return 0;
}
```

## 7.8 Convert from base to another

```cpp
// Convert Decimal to any base

string decimal_to_any_base(ll decimal, ll base){
    if(decimal == 0) return "0";
    string num = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    string result;
    do{
        result.push_back(num[decimal % base]);
        decimal /= base;
    }while(decimal != 0);
    return string(result.rbegin(), result.rend());
}

// Convert any base to decimal
ll  any_base_to_decimal(string str, int base) {
    auto val = [](char c){
        return (c >= '0' && c <= '9' ? (int) c - '0' : (int) c - 'A' + 10);
    };
    ll len = sz(str), power = 1, num = 0, i;
    for (i = len - 1; i >= 0; i--) {
        num += val(str[i]) * power;
        power = power * base;
    }
    return num;
}
```

### 7.9 Halazona

```cpp
vector < int > Halazona(vector < vector < int > > matrix) {
    vector < int > result;
    int rows = matrix.size(), cols = matrix[0].size();
    int left = 0, right = cols-1, top = 0, bottom = rows-1;

    while (left <= right and top <= bottom) {
        for (int i = left; i <= right; i++) result.push_back(matrix[top][i]);
        top++;

        for (int i = top; i <= bottom; i++)
            result.push_back(matrix[i][right]);
        right--;

        if (top <= bottom) {
            for (int i = right; i >= left; i--)
                result.push_back(matrix[bottom][i]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--)
                result.push_back(matrix[i][left]);
            left++;
        }
    }
    return result;
}

vector < vector < int > > generate_halazona(int n) {
        vector<vector<int>> matrix(n, vector<int>(n, 0));
        int left = 0, right = n-1, top = 0, bottom = n-1, num = 1;
        while (left <= right && top <= bottom) {
```

```
            for (int i = left; i <= right; i++) matrix[top][i] = num++;
            top++;
            for (int i = top; i <= bottom; i++) matrix[i][right] = num++;
            right--;
            if (top <= bottom) {
                for (int i = right; i >= left; i--) matrix[bottom][i] =
                    num++;
                bottom--;
            }
            if (left <= right) {
                for (int i = bottom; i >= top; i--) matrix[i][left] = num++;
                left++;
            }
        }
        return matrix;
}
```

### 7.10 Grundy with fixed moves

```
struct grundy {
    private:
    int prefix = -1, cycle = -1;
    int size;
    vector<int> Grundy;

    void computeGrundy(vector<int>&moves) {
        for (int i = 0; i < size; i++) {
            set<int> next;
            for (int m : moves) {
                if (i >= m)
                    next.insert(Grundy[i - m]);
            }
            int g = 0;
            while (next.count(g)) g++;
            Grundy[i] = g;
        }
    }
    void find_cycle() {
        static bool done = false;
        if (done) return;
        done = true;
        for (int len = 2; len <= size; len++) {
            for (int start = 2; start < size - len; start++) {
                bool found = true;
                for (int i = 0; i < len; i++) {
                    if (Grundy[start + i] != Grundy[start + len + i]) {
                        found = false;
                        break;
                    }
                }
                if (found) {
                    prefix = start;
                    cycle = len;
                    return;
                }
            }
        }
    }
    public:
    grundy(int max, vector<int>moves) {
```

```
        size = max;
        Grundy.resize(max);
        computeGrundy(moves);
    }
    int compute(ll val) {
        if (val < prefix + cycle) return Grundy[val];
        find_cycle();
        return Grundy[prefix + (val - prefix) % cycle];
    }

};
```

# 8 Tricks

## 8.1 XOR numbers from 1 to R

```
// XOR from 1 to R
ll XOR_UptoR(ll R) {
    ll res = 0, i;
    for (i = R - (R % 4); i <= R; i++)
        res ^= i;
    return res;
}
```

## 8.2 XOR from 1 to $R$, skipping numbers where $N \bmod 2^x \neq y$

```
// XOR from 1 to R, skipping numbers where N % 2^x != y
ll XOR_UptoR_SkipModCondition(ll R, ll x, ll y) {
    if (R < y) return 0;
    ll k = ((R - y) >> x);
    return (XOR_UptoR(k) << x) ^ (y * (!(k & 1)));
}
```

## 8.3 XOR from 1 to $R$ with inc $X$

```
// XOR from L to R, considering increments of x (x must be a power of two)
ll XOR_FromLtoR_WithIncrement(ll L, ll R, ll x) {
    ll md = L % x;
    x = log2l(x);
    return XOR_UptoR_SkipModCondition(R, x, md) ^
       XOR_UptoR_SkipModCondition(L - 1, x, md);
}
```

## 8.4 AND For range L to R

```
ll msb(ll n)
{
    ll msb=-1;
    while(n>0)
    {
        n=n/2;
        msb++;
    }
    return msb;
}
//bitwise and of a range datatype ll
```

```
ll andRange(ll l,ll r){
    ll ans=0;
    for(ll i=msb(r);i>=0;i--)
    {
        ll la = 1;
        if((l&(la<<i))==(r&(la<<i)))
        {
            ans+=(l&(la<<i));
        }
        else
        {
            break;
        }
    }
    return ans;
}
```

### 8.5 Random number in Range

```
std::mt19937_64
   rng(std::chrono::system_clock::now().time_since_epoch().count());
ll rand(ll l,ll r)
{
    return uniform_int_distribution<ll>(l, r)(rng);
}
```

### 8.6 Base-K Mask

```
struct BaseMask {
    int base;
    BaseMask(int base) : base(base) {}
    // Get the digit at index idx in the base representation of mask
    int get(int mask, int idx) const {
        for (int i = 0; i < idx; ++i) {
            mask /= base;
        }
        return mask % base;
    }
    // Set the digit at index idx to val in the base representation of mask
    int put(int mask, int idx, int val) const {
        int powB = 1;
        for (int i = 0; i < idx; ++i) powB *= base;
        int cur = (mask / powB) % base;
        return mask - cur * powB + val * powB;
    }
};
```

### 8.7 Generate Formula for $\sum_{i=1}^{n} i^k$

```
from fractions import Fraction

def factorial(n):
    result = 1
    for i in range(2, n+1):
        result *= i
    return result

def binomial(n, k):
```

```
    return factorial(n) // (factorial(k) * factorial(n - k))

def bernoulli_numbers(n):
    A = [Fraction(0)] * (n + 1)
    B = []

    for m in range(n + 1):
        A[m] = Fraction(1, m + 1)
        for j in range(m, 0, -1):
            A[j - 1] = j * (A[j - 1] - A[j])
        B.append(A[0])
    return B

def power_sum_formula(k):
    B = bernoulli_numbers(k)
    terms = []

    for j in range(k + 1):
        bj = B[j]
        if bj == 0:
            continue

        binom = binomial(k + 1, j)
        coeff = Fraction(binom) * bj / Fraction(k + 1)

        power = k + 1 - j
        if coeff == 0:
            continue

        if coeff.denominator == 1:
            coeff_str = f"{coeff.numerator}" if abs(coeff.numerator) != 1
                else ("-" if coeff.numerator < 0 else "")
        else:
            coeff_str = f"({coeff})"

        if power == 0:
            term = f"{coeff_str}"
        elif power == 1:
            term = f"{coeff_str}*n"
        else:
            term = f"{coeff_str}*n^{power}"

        terms.append(term)

    equation = " + ".join(terms).replace("+ -", "- ")
    return f"sum(i^({k}), i=1..n) = {equation}"

def evaluate_sum(k, n):
    B = bernoulli_numbers(k)
    total = Fraction(0)

    for j in range(k + 1):
        bj = B[j]
        if bj == 0:
            continue
        binom = binomial(k + 1, j)
        coeff = Fraction(binom) * bj / Fraction(k + 1)
        total += coeff * n**(k + 1 - j)
    return total

# Example:
```

```
k = 5
n = 20

equation = power_sum_formula(k)
value = evaluate_sum(k, n)

print(equation)
print(f"For n = {n}, the sum is {value} or approx {float(value):.6f}")
```

# 9 Summation Formulas

## 9.1 Natural Number Sums

$$\sum_{i=1}^{n} 1 = n$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$$

$$\sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\sum_{i=1}^{n} i^k = \frac{1}{k+1}\sum_{j=0}^{k}\binom{k+1}{j}B_j n^{k+1-j}$$

## 9.2 Even & Odd Number sums

$$\sum_{i=1}^{n} 2i = n(n+1)$$

$$\sum_{i=1}^{n} (2i-1) = n^2$$

## 9.3 Arithmetic Sequence Sum

$$\sum_{i=1}^{n} [a + (i-1)d] = \frac{n}{2}[2a + (n-1)d]$$

## 9.4 Geometric Sequence Sum

$$\sum_{i=0}^{n-1} ar^i = \frac{a(1-r^n)}{1-r} \quad \text{for } r \neq 1$$

$$\sum_{i=0}^{\infty} ar^i = \frac{a}{1-r} \quad \text{for } |r| < 1$$

$$\sum_{i=1}^{n} k^i = \begin{cases} \frac{k(k^n-1)}{k-1}, & \text{if } k \neq 1 \\ n, & \text{if } k = 1 \end{cases}$$

$$\sum_{i=1}^{n} i \cdot k^i = \begin{cases} \frac{k\left(1-(n+1)k^n+nk^{n+1}\right)}{(1-k)^2}, & \text{if } k \neq 1 \\ \frac{n(n+1)}{2}, & \text{if } k = 1 \end{cases}$$

# Binomial Coefficients and Lucas's Theorem

## Lucas's Theorem

If $p$ is a prime:

$$\binom{n}{r} \bmod p = \binom{n \bmod p}{r \bmod p} \cdot \binom{\left\lfloor \frac{n}{p} \right\rfloor}{\left\lfloor \frac{r}{p} \right\rfloor} \bmod p$$

## Basic Binomial Identities

$$\sum_{r=0}^{n} \binom{n}{r} = 2^n$$

$$\sum_{r=0}^{n} (-1)^r \binom{n}{r} = 0$$

$$\sum_{r=0}^{n} \binom{n}{r}^2 = \binom{2n}{n}$$

$$\sum_{r=0}^{n} r\binom{n}{r} = n \cdot 2^{n-1}$$

$$\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$$

$$\sum_{i=k}^{n} \binom{i}{k} = \binom{n+1}{k+1}$$